SESHADRI RAO GUDLAVALLERU ENGINEERING COLLEGE

Academics Strengthening & Advancement (AS&A)

Department of AI & ML

R – 23

II B.Tech. I Semester

Learning Material

On

ARTIFICIAL INTELLIGENCE

UNIT – II

"Searching"

Prepared by : ***Dr.*** **R. Abinaya**

Asst. Prof.,

CSE Dept.

SRGEC, Gudlavalleru

SYLLABUS

ARTIFICIAL INTELLIGENCE

II Year I Semester

Lecture: 3 Internal Marks: 30
Credits: 3 External Marks: 70

Pre-requisite:

1. Knowledge in Computer Programming.
2. A course on "Mathematical Foundations of Computer Science".
3. Background in linear algebra, data structures and algorithms, and probability.

Course Objectives:

- The student should be made to learn the methods of solving problems using Artificial Intelligence.
- The student should be made to introduce the concepts of Expert Systems.

Course Outcomes:

Upon successful completion of the course, the students will be able to

- Formulate problems using Artificial Intelligence representations.
- Evaluate the effectiveness of heuristic search and game playing algorithms in different domains.
- Design and implement knowledge bases that combine logical, semantic, and probabilistic representations
- Apply resolution and explanation-based learning techniques to derive conclusions from logical premises.
- Analyze case studies of prominent expert systems like MYCIN, DART, and XCON.

Course Content:

UNIT - I

Introduction: AI problems, foundation of AI and history of AI, intelligent agents: Agents and Environments, the concept of rationality, the nature of environments, structure of agents, problem solving agents, problem formulation.

UNIT - II

Searching- Searching for solutions, uniformed search strategies – Breadth first search, Depth first Search. Search with partial information (Heuristic search) Hill climbing, A*, AO* Algorithms, Problem reduction, Game Playing-Adversial search, Games, mini-max algorithm, optimal decisions in multiplayer games, Problem in Game playing, Alpha-Beta pruning, Evaluation functions.

UNIT - III

Representation of Knowledge: Knowledge representation issues, predicate logic- logic programming, semantic nets- frames and inheritance, representing knowledge using rules. Reasoning under uncertainty, review of probability, Bayes' probabilistic interferences and dempster shafer theory.

UNIT - IV

Logic concepts: First order logic. Inference in first order logic, propositional vs. first order inference, unification & lifts, Forward chaining, Backward chaining, Resolution, Learning from observation Inductive learning, Decision trees, Explanation based learning, Reinforcement Learning.

UNIT - V

Expert Systems: Architecture of expert systems, Roles of expert systems – Knowledge Acquisition Meta knowledge Heuristics. Typical expert systems – MYCIN, DART, XCON: Expert systems shells.

Textbooks:

1. S. Russel and P. Norvig, "Artificial Intelligence – A Modern Approach", Second Edition, Pearson Education.
2. Kevin Night and Elaine Rich, Nair B., "Artificial Intelligence (SIE)", Mc Graw Hill

Reference Books:

1. David Poole, Alan Mackworth, Randy Goebel, "Computational Intelligence: a logical approach", Oxford University Press.
2. G. Luger, "Artificial Intelligence: Structures and Strategies for complex problem solving", Fourth Edition, Pearson Education.
3. J. Nilsson, "Artificial Intelligence: A new Synthesis", Elsevier Publishers.
4. Artificial Intelligence, Saroj Kaushik, Cengage Learning.
5. Donald A. Waterman, "A Guide to Expert Systems", Pearson

Online Learning Resources:

1. https://ai.google/
2. https://swayam.gov.in/nd1_noc19_me71/preview

LEARNING MATERIAL

UNIT-II

SEARCHING

Objective:

- To familiarize with the basic principles of AI towards problem solving, inference, and learning.

Syllabus:

Searching- Searching for solutions, uniformed search strategies – Breadth first search, Depth first Search. Search with partial information (Heuristic search) Hill climbing, A*, AO* Algorithms, Problem reduction, Game Playing-Adversial search, Games, mini-max algorithm, optimal decisions in multiplayer games, Problem in Game playing, Alpha-Beta pruning, Evaluation functions.

Learning Outcomes:

At the end of the unit, students will be able to

- differentiate informed search and Uninformed Search mechanism.
- implement game Playing.

Module 1: Uninformed Search

Module 2: Informed Search

Module 3: Game Playing

1. SEARCHING FOR SOLUTIONS

- A search algorithm takes a search problem as input and returns a solution, or an indication of failure.
- These algorithms that superimpose a search tree over the state-space graph, forming various paths from the initial state, trying to find a path that reaches a goal state.
- Each node in the search tree corresponds to a state in the state space and the edges in the search tree correspond to actions. The root of the tree corresponds to the initial state of the problem.
- It is important to understand the distinction between the state space and the search tree.
- The state space describes the (possibly infinite) set of states in the world, and the actions that allow transitions from one state to another.
- The search tree describes paths between these states, reaching towards the goal.

- The search tree may have multiple paths to (and thus multiple nodes for) any given state, but each node in the tree has a unique path back to the root (as in all trees).
- Figure 2.1 below shows the first few steps in finding a path from Arad to Bucharest.
- Nodes that have been expanded are lavender with bold letters; nodes on the frontier that have been generated but not yet expanded are in green; the set of states corresponding to these two types of nodes are said to have been reached. Nodes that could be generated next are shown in faint dashed lines. Notice in the bottom tree there is a cycle from Arad to Sibiu to Arad; that can't be an optimal path, so search should not continue from there.
- The root node of the search tree is at the initial state, Arad.
- We can expand the node, by considering the available ACTIONS for that state, using the RESULT function to see where those actions lead to, and generating a new node (called a child node or successor node) for each of the resulting states.
- Each child node has Arad as its parent node.
- Now we must choose which of these three child nodes to consider next. This is the essence of search—following up one option now and putting the others aside for later. Suppose we choose to expand Sibiu first.
- Figure 2.1 (bottom) shows the result: a set of 6 unexpanded nodes (outlined in bold). We call this the frontier of the search tree. We say that any state that has had a node generated for it has been reached (whether or not that node has been expanded).

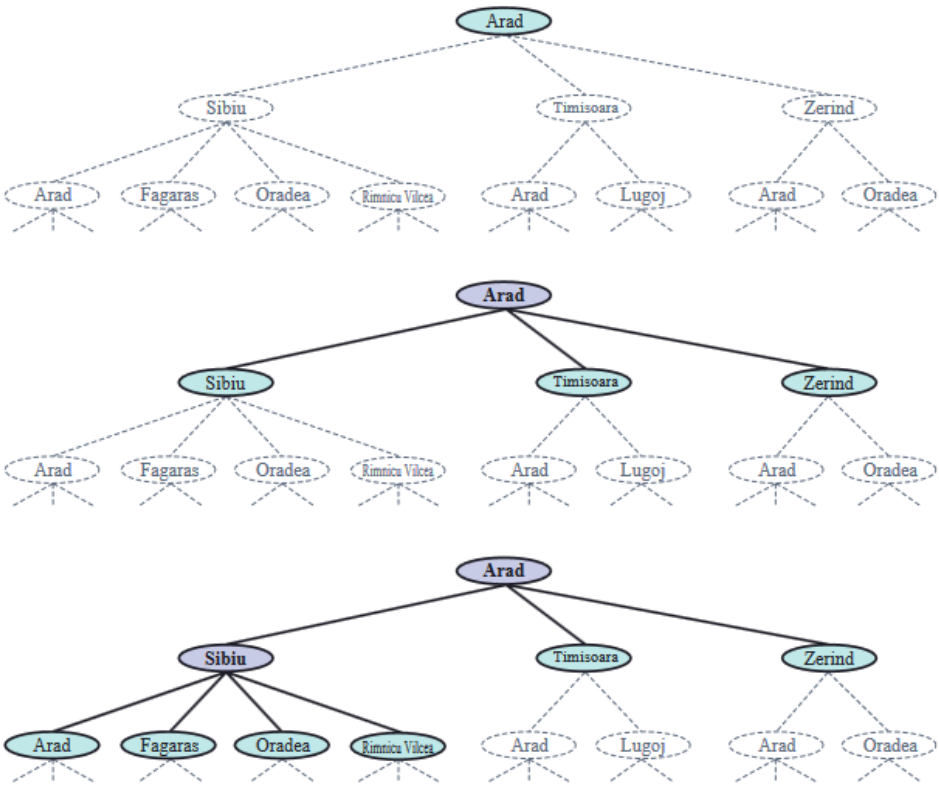


Figure 2.1: Three partial search trees for finding a route from Arad to Bucharest.

- Based on the search problems we can classify the search algorithms into uninformed (Blind search) search and informed search (Heuristic search) algorithms as shown in the following figure 2.

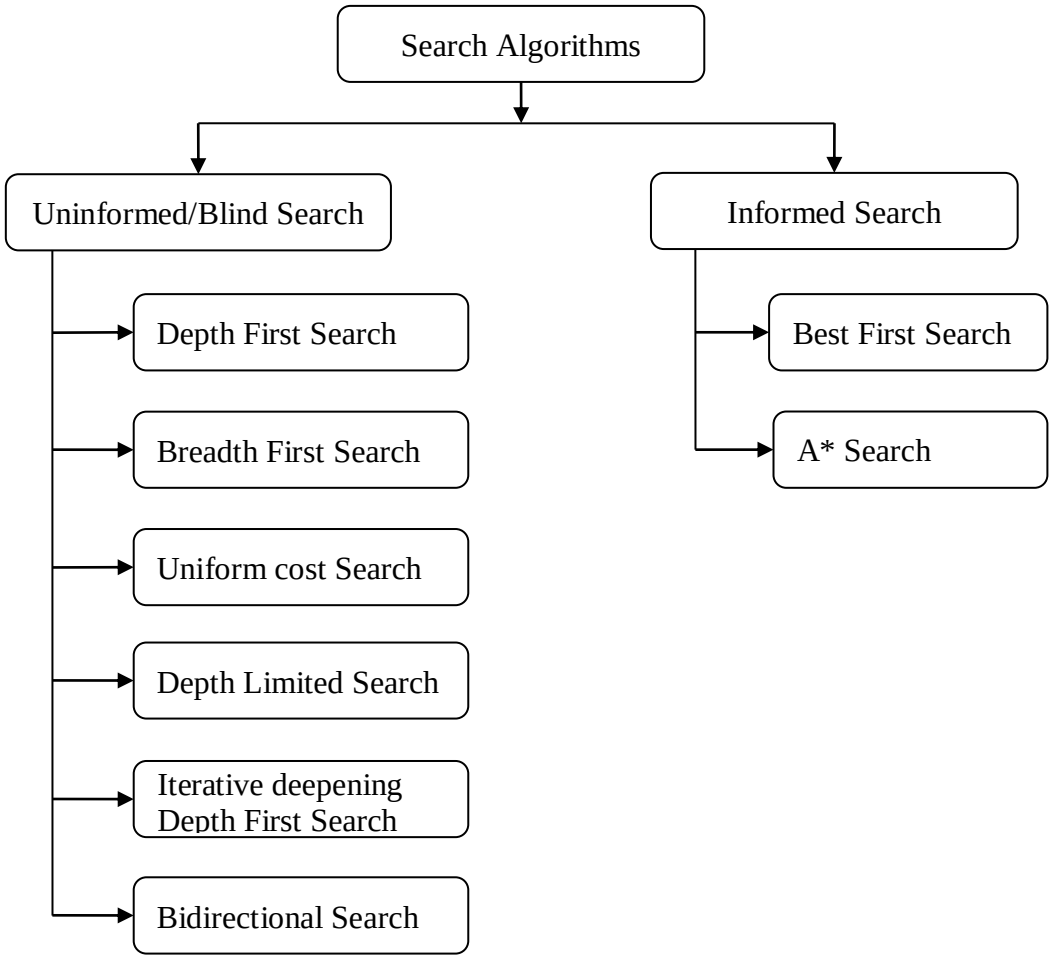


Figure 2.2: Classification of search algorithms

1.1 Uninformed/Blind Search

- The uninformed search does not contain any domain knowledge such as closeness, the location of the goal.
- It operates in a brute-force way as it only includes information about how to traverse the tree and how to identify leaf and goal nodes.
- Uninformed search applies a way in which search tree is searched without any information about the search space like initial state operators and test for the goal, so it is also called blind search.
- It examines each node of the tree until it achieves the goal node.
- It can be divided into five main types.

1. Breadth-first search
2. Uniform cost search
3. Depth-first search
4. Iterative deepening depth-first search
5. Bidirectional Search

1.2 Informed Search

- Informed search algorithms use domain knowledge.
- In an informed search, problem information is available which can guide the search.
- Informed search strategies can find a solution more efficiently than an uninformed search strategy.
- Informed search is also called a Heuristic search.
- A heuristic is a way which might not always be guaranteed for best solutions but guaranteed to find a good solution in reasonable time.

2. DEPTH-FIRST SEARCH

- Depth-first search always expands the *deepest* node in the current frontier of the search tree.
- The progress of the search is illustrated in Figure 2.3.
- The search proceeds immediately to the deepest level of the search tree, where the nodes have no successors.
- As those nodes are expanded, they are dropped from the frontier, so then the search "backs up" to the next deepest node that still has unexplored successors.

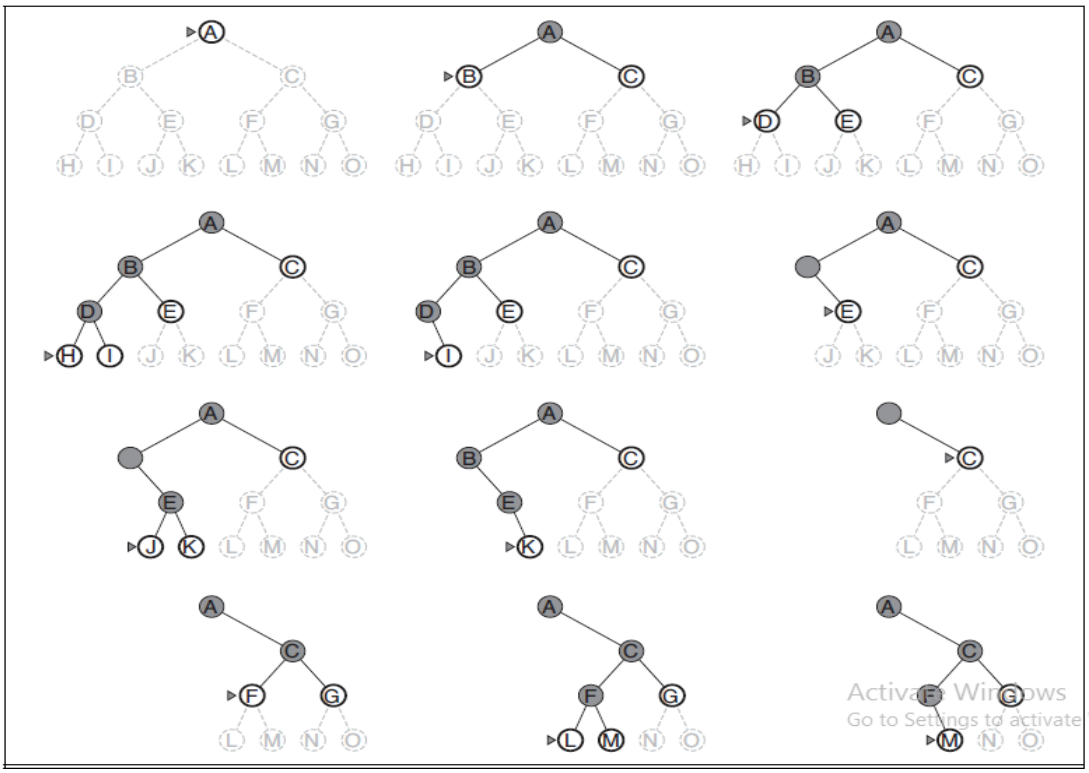
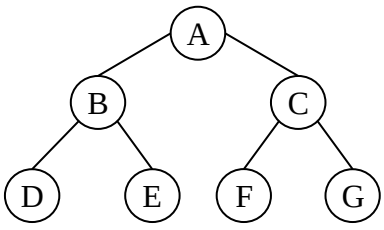


Figure 2.3: Depth-first search on a binary tree. The unexplored region is shown in light gray. Explored nodes with no descendants in the frontier are removed from memory. Nodes at depth 3 have no successors and M is the only goal node.

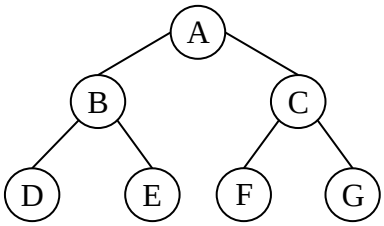
Example 1: Search for the node E in the tree/graph structure with the help of Depth-First Search using Open and Closed list.



Open List	Closed List
A	---
B, C	A
D, E, C	A, B
E, C	A, B, D
C	A, B, D, E

- Depth-First Search use STACK data structure. So Open List in DFS is **STACK**.
- Here goal state E is found. So search becomes Success

Example 2: Search for the node H in the tree/graph structure with the help of Depth-First Search using Open and Closed list.



Open List	Closed List
A	---
B, C	A
D, E, C	A, B
E, C	A, B, D
C	A, B, D, E
F, G	A, B, D, E, C
G	A, B, D, E, C, F
---	A, B, D, E, C, F, G

- In the above example, Open list become empty and till the goal state is not found. So search becomes failure.

Algorithm: Depth-First Search

1. If the initial state is a goal state, quit and return success.

2. Otherwise, do the following until success or failure is returned:

   (a) Generate a successor, *E*, of the initial state. If there are no more successors, return failure.

   (b) Call Depth-First Search with *E* as the initial state.

   (c) If success is returned, quit. Otherwise continue in this loop

2.1 Advantage

1. DFS requires very less memory as it only needs to store a stack of the nodes on the path from root node to the current node.
2. It takes less time to reach to the goal node than BFS algorithm (if it traverses in the right path.

2.2 Disadvantage

1. There is the possibility that many states keep re-occurring, and there is no guarantee of finding the solution.
2. DFS algorithm goes for deep down searching and sometime it may go to the infinite loop.

3. BREADTH-FIRST SEARCH

- Breadth-first search is the most common search strategy for traversing a tree or graph. This algorithm searches breadth (level) wise in a tree or graph, so it is called breadth-first search.
- BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to nodes of next level.
- The breadth-first search algorithm is an example of a general-graph search algorithm.
- Breadth-first search implemented using FIFO queue data structure.

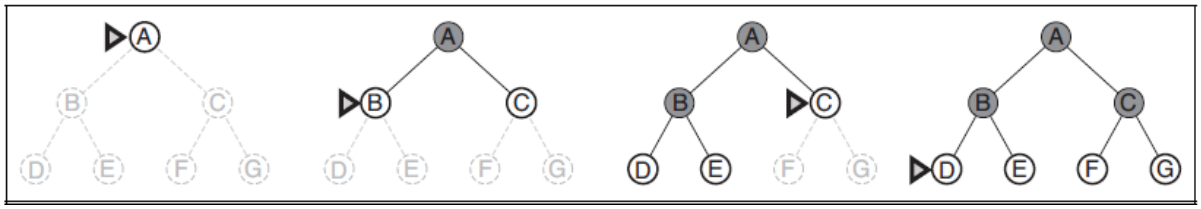
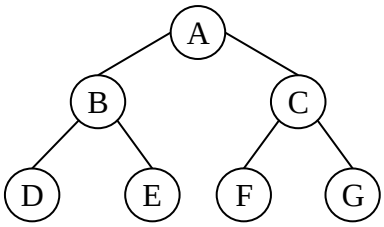


Figure 2.4: Breadth-first search on a simple binary tree. At each stage, the node to be expanded next is indicated by a marker.

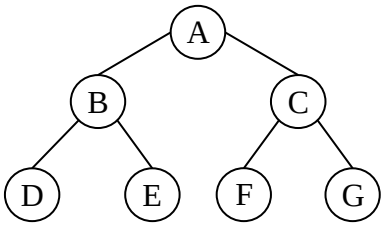
Example 1: Search for the node E in the tree/graph structure with the help of Breadth-First Search using Open and Closed list.



Open List	Closed List
A	---
B, C	A
C, D, E	A, B
D, E, F, G	A, B, C
E, F, G	A, B, C, D
F, G	A, B, C, D, E

- Here goal state E is found. So search becomes Success.
- Breadth-First Search use QUEUE data structure. So Open List in BFS is **QUEUE**.

Example 2: Search for the node H in the tree/graph structure with the help of Breadth-First Search using Open and Closed list.



Open List	Closed List
A	---
B, C	A
C, D, E	A, B
D, E, F, G	A, B, C
E, F, G	A, B, C, D
F, G	A, B, C, D, E
G	A, B, C, D, E, F
---	A, B, C, D, E, F, G

- In the above example, Open list become empty and till the goal state is not found. So search becomes failure.

Algorithm: Breadth-First Search

1. Create a variable called *NODE-LIST* and set it to the initial state.

2. Until a goal state is found or *NODE-LIST* is empty:

   (a) Remove the first element from *NODE-LIST* and call it *E*. If *NODE-LIST* was empty, quit.

   (b) For each way that each rule can match the state described in *E* do:

      (i) Apply the rule to generate a new state.

      (ii) If the new state is a goal state, quit and return this state.

      (iii) Otherwise, add the new state to the end of *NODE-LIST*.

3.1 Advantages

1. BFS will provide a solution if any solution exists.
2. If there are more than one solutions for a given problem, then BFS will provide the minimal solution which requires the least number of steps.

3.2 Disadvantage

1. It can be slow since it expands all the nodes at each level before moving on to the next level.
2. Memory Constraints as it stores all the nodes of the present level to go for the next level.

Module 2: Informed Search

4. HEURISTIC SEARCH

- Heuristic search is also termed as an informed search strategy one that uses problem-specific knowledge beyond the definition of the problem itself can find solutions more efficiently than can an uninformed strategy.
- *A heuristic* is a technique that improves the efficiency of a search process, possibly by sacrificing claims of completeness.
- Using good heuristics, we can hope to get good solutions to hard problems, such as the traveling salesman, in less than exponential time.
- There are some good general purpose heuristics that are useful in a wide variety of problem domains.
- One example of a good general-purpose heuristic that is useful for a variety of combinatorial problems is the *nearest neighbor heuristic,* which works by selecting the locally superior alternative at each step. Applying it to the traveling salesman problem, we produce the following procedure:

  1. Arbitrarily select a starting city.

  2. To select the next city, look at all cities not yet visited, and select the one closest to the current city. Go to it next.

  3. Repeat step 2 until all cities have been visited.
- Most search algorithms include as a component of heuristic function, denoted h(n).

5. HILL CLIMBING ALGORITHM

- Hill climbing algorithm is a local search algorithm which continuously moves in the direction of increasing elevation/value to find the peak of the mountain or best solution to the problem. It terminates when it reaches a peak value where no neighbor has a higher value.
- Hill climbing algorithm is a technique which is used for optimizing the mathematical problems. One of the widely discussed examples of Hill climbing algorithm is Traveling-salesman Problem in which we need to minimize the distance traveled by the salesman.
- It is also called greedy local search as it only looks to its good immediate neighbor state and not beyond that.
- A node of hill climbing algorithm has two components which are state and value.
- Hill Climbing is mostly used when a good heuristic is available.
- In this algorithm, we don't need to maintain and handle the search tree or graph as it only keeps a single current state.

5.1 Simple Hill Climbing

- Simple hill climbing is the simplest way to implement a hill climbing algorithm.
- It only evaluates the neighbor node state at a time and selects the first one which optimizes current cost and set it as a current state.
- It only checks it's one successor state, and if it finds better than the current state, then move else be in the same state. This algorithm has the following features:
- Less time consuming.
- Less optimal solution and the solution is not guaranteed

Algorithm for Simple Hill Climbing

Step 1: Evaluate the initial state, if it is goal state then return success and Stop.

Step 2: Loop Until a solution is found or there is no new operator left to apply.

Step 3: Select and apply an operator to the current state.

Step 4: Check new state:

If it is goal state, then return success and quit.

Else if it is better than the current state then assign new state as a current state.

Else if not better than the current state, then return to step2.

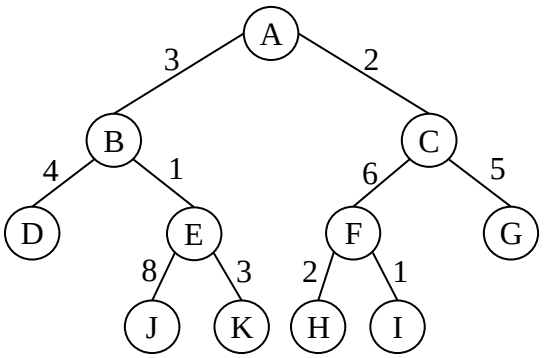
Step 5: Exit.

6. BEST-FIRST SEARCH

- Best First Search, which is a way of combining the advantages of both Depth First Search and Breadth First Search into a single method.
- Best First Search algorithm always selects the path which appears best at that moment.
- It uses the heuristic function and search.
- With the help of Best First Search, at each step, we can choose the most promising node.
- In the Best First Search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function h(n).
- Where, h(n) is estimated cost from node n to the goal.
- The Best First algorithm is implemented by the **priority queue**.
- The parent link will make it possible to recover the path to the goal once the goal is found. The list of successors will make it possible, if a better path is found to an already existing node, to propagate the improvement down to its successors. We will

call a graph of this sort an ***OR graph***, since each of its branches represents an alternative problem-solving path.

Example 1: Search for node K in the following Tree/Graph using Best First Search with the help of Open and Closed List.

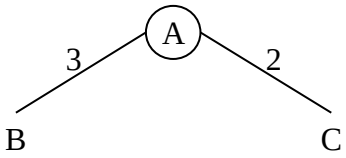


Open List	Closed List
A_0	---
B_3, C_2	A_0
B_3, F_8, G_7	A_0, C_2
F_8, G_7, D_7, E_4	A_0, C_2, B_3
F_8, G_7, D_7, J_{12}, K_7	A_0, C_2, B_3, E_4
F_8, D_7, J_{12}, K_7	A_0, C_2, B_3, E_4, G_7
F_8, J_{12}, K_7	A_0, C_2, B_3, E_4, G_7, D_7
F_8, J_{12}	A_0, C_2, B_3, E_4, G_7, D_7, K_7

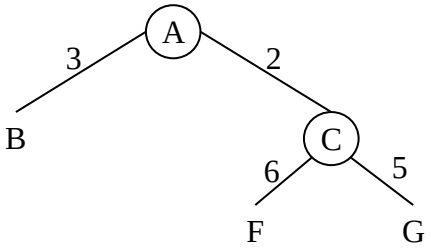
- Here node K is GOAL state. So Search becomes Success.
- Final Path Cost is 7 (3 + 1 +3)

Initially 

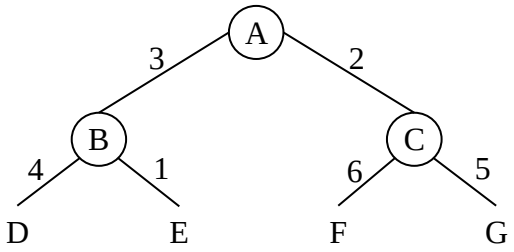
- Current node A is not a GOAL state. So expand node A.



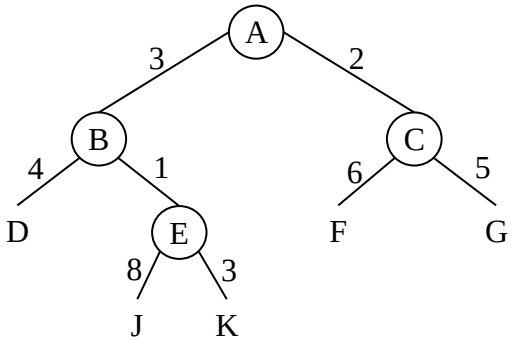
- Now node C becomes Best node. Here current node C is not a GOAL state. So expand node C.



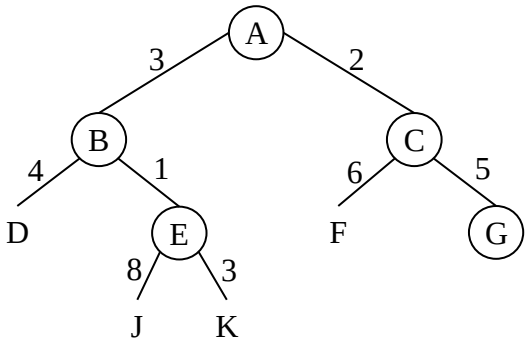
- Now node B becomes Best node. Here current node B is not a GOAL state. So expand node B.



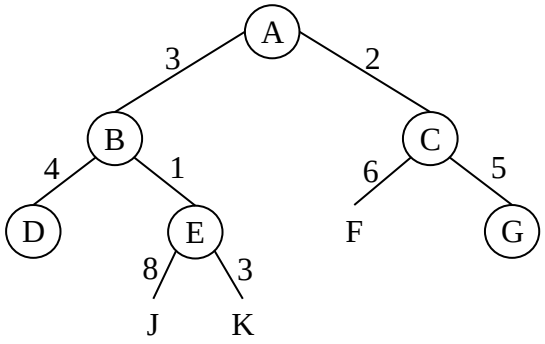
- Now node E becomes Best node. Here current node E is not a GOAL state. So expand node E.



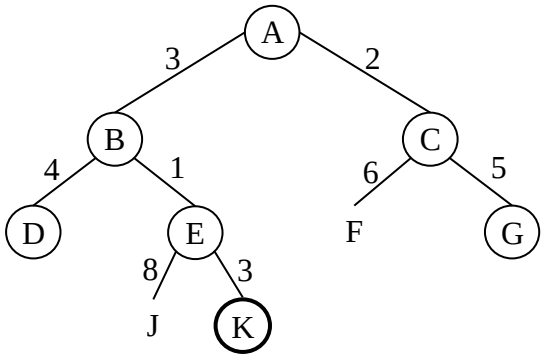
- Now node G becomes Best node. Here current node G is not a GOAL state. So expand node G.



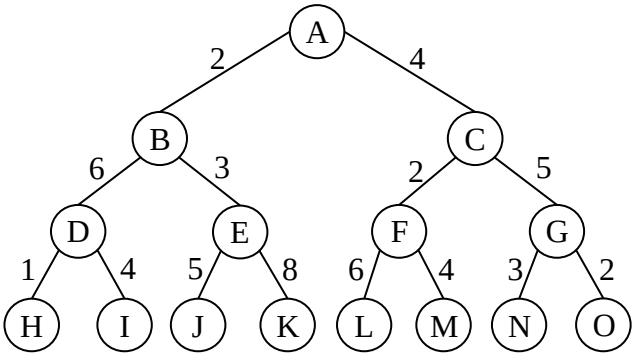
- Now node D becomes Best node. Here current node D is not a GOAL state. So expand node D.



- Now node K becomes Best node. Here current node K is GOAL state. So stop the search process.



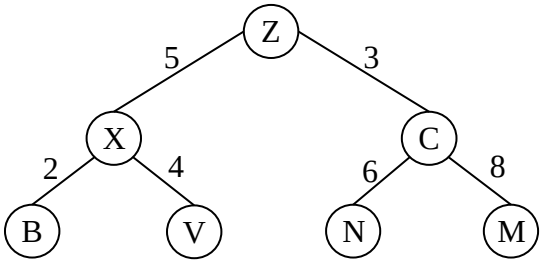
Example 2: Search for node F in the following Tree/Graph using Best First Search with the help of Open and Closed List.



Open List	Closed List
A_0	---
B_2, C_4	A_0
C_4, D_8, E_5	A_0, B_2
D_8, E_5, F_6, G_9	A_0, B_2, C_4
D_8, F_6, G_9, J_{10}, K_{13}	A_0, B_2, C_4, E_5
D_8, G_9, J_{10}, K_{13}, L_{12}, M_{10}	A_0, B_2, C_4, E_5, F_6,

- Here node **F** is GOAL state. So Search becomes Success.
- Final Path Cost is 6 (4 + 2)

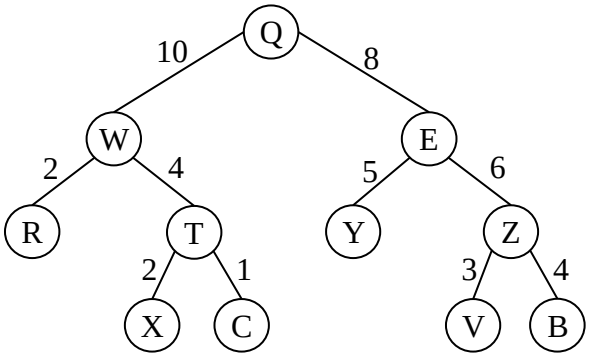
Example 3: Search for node **A** in the following Tree/Graph using Best First Search with the help of Open and Closed List.



Open List	Closed List
Z_0	---
X_5, C_3	Z_0
X_5, N_9, M_{11}	Z_0, C_3
N_9, M_{11}, B_7, V_9	Z_0, C_3, X_5
N_9, M_{11}, V_9	Z_0, C_3, X_5, B_7
M_{11}, V_9	Z_0, C_3, X_5, B_7, N_9
M_{11}	Z_0, C_3, X_5, B_7, N_9, V_9
---	Z_0, C_3, X_5, B_7, N_9, V_9, M_{11}

- Here Open list becomes empty and goal state is not found. So search becomes failure.

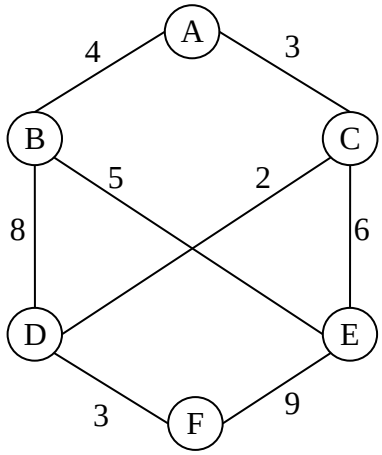
Example 4: Search for node **Y** in the following Tree/Graph using Best First Search with the help of Open and Closed List.



Open List	Closed List
Q_0	---
W_{10}, E_8	Q_0
W_{10}, Y_{13}, Z_{14}	Q_0, E_8
Y_{13}, Z_{14}, R_{12}, T_{14}	Q_0, E_8, W_{10},
Y_{13}, Z_{14}, T_{14},	Q_0, E_8, W_{10}, R_{12}
Z_{14}, T_{14}	Q_0, E_8, W_{10}, R_{12}, Y_{13}

- Here node **Y** is GOAL state. So Search becomes Success.
- Final Path Cost is 13 (8 + 5)

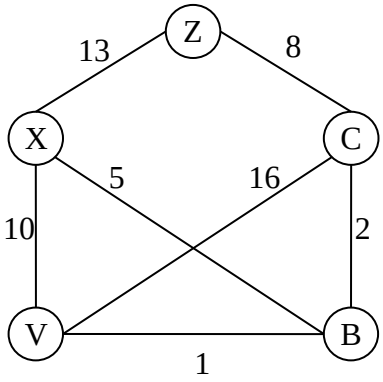
Example 5: Search for node **F** in the following Tree/Graph using Best First Search with the help of Open and Closed List.



Open List	Closed List
A_0	---
B_4, C_3	A_0
B_4, D_5, E_9	A_0, C_3
D_5, E_9	A_0, C_3, B_4
E_9, F_8	A_0, C_3, B_4, D_5
E_9	A_0, C_3, B_4, D_5, F_8

- Here node **F** is GOAL state. So Search becomes Success.
- Final Path Cost is 13 (3 + 2 +3)

Example 6: Search for node **V** in the following Tree/Graph using Best First Search with the help of Open and Closed List.



Open List	Closed List
Z_0	---
X_{13}, C_8	Z_0
X_{13}, V_{24}, B_{10}	Z_0, C_8
X_{13}, V_{11}	Z_0, C_8, B_{10}
X_{13},	Z_0, C_8, B_{10}, V_{11}

- Here node **V** is GOAL state. So Search becomes Success.
- Final Path Cost is 11 (8 + 2 +1)

Initially



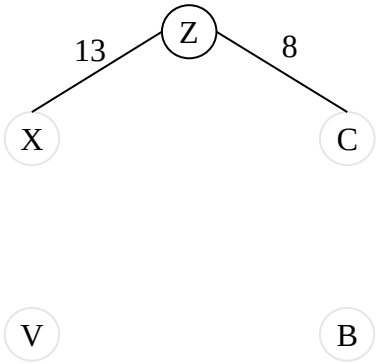




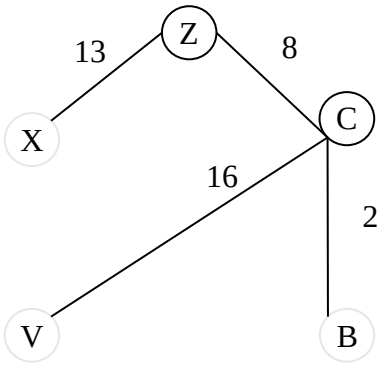




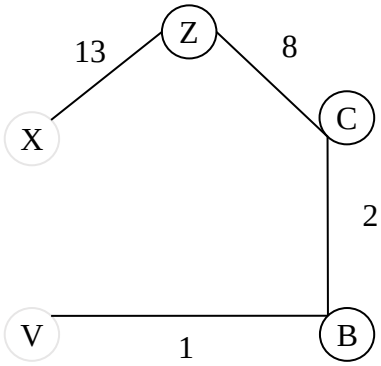
- Here current node **Z** is not a GOAL state. So expand node Z.



- Now node C becomes Best node. Here current node C is not a GOAL state. So expand node C.

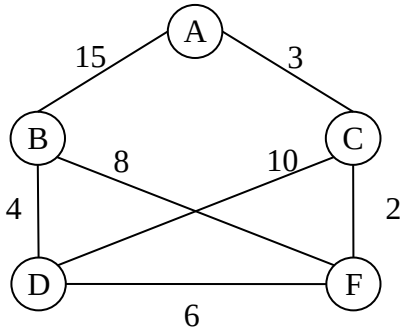


- Now node B becomes Best node. Here current node B is not a GOAL state. So expand node C.



- Here node V obtain better path through node B. So change parent of node V as B.
- Now node V becomes Best node. Here current node V is a GOAL state. So examine node C.

Example 7: Search for node **D** in the following Tree/Graph using Best First Search with the help of Open and Closed List.



Open List	Closed List
A_0	---
B_{15}, C_3	A_0
B_{15}, D_{13}, F_5	A_0, C_3
B_{13}, D_{11}	A_0, C_3, F_5
B_{13}	A_0, C_3, F_5, D_{11}

- Here node **D** is GOAL state. So Search becomes Success.
- Final Path Cost is 11 (3 + 2 + 6)

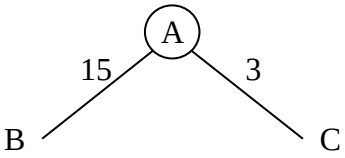
Initially



B C

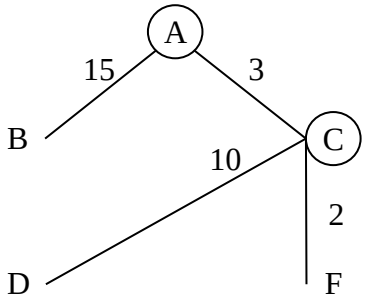
D F

- Here current node **A** is not a GOAL state. So expand node A.

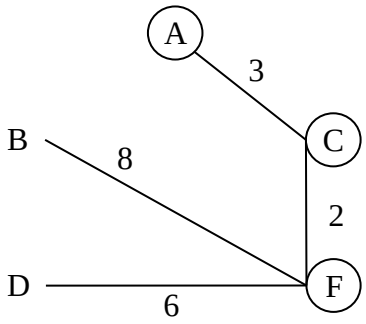


D F

- Now node C becomes Best node. Here current node C is not a GOAL state. So expand node C.



- Now node F becomes Best node. Here current node F is not a GOAL state. So expand node F.



- Here best path cost is calculated from node A to B, A to D through node F. So parent of B and D is changed to node F.
- Now node D becomes Best node. Here current node D is a GOAL state. So success.

Algorithm: Best-First Search

1. Start with *OPEN LIST* containing just the initial state.

2. Until a goal is found or there are no nodes left on *OPEN LIST* do:

   (a) Remove the node **n**, from the OPEN LIST which has the lowest value of h(n)

   (b) if node **n** is goal state, stop the search process and return node **n**

   (c) else generate successors of node **n**

   (d) For each successor do:

      (i) If it has not been generated before, evaluate it, add it to *OPEN LIST*, and record its parent.

      (ii) If it has been generated before, change the parent if this new path is better than the previous one. In that case, update the cost of getting to this node and to any successors that this node may already have.

6.1 Advantages

- Best first search can switch between BFS and DFS by gaining the advantages of both the algorithms.
- This algorithm is more efficient than BFS and DFS algorithms.

6.2 Disadvantages

- It can behave as an unguided depth-first search in the worst case scenario.
- It can get stuck in a loop as DFS.
- This algorithm is not optimal.

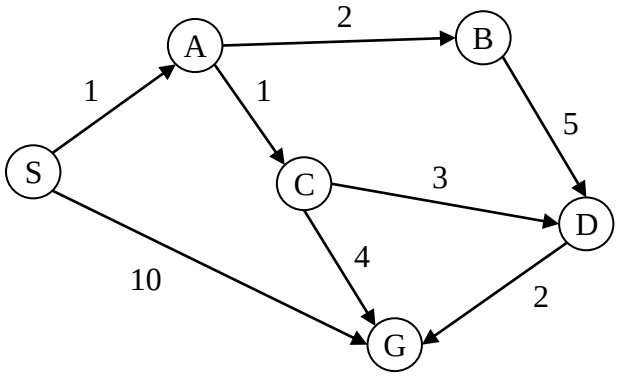
7. THE A* ALGORITHM

- The best-first search algorithm that was just presented is a simplification of an algorithm called A*.
- This algorithm uses *OPEN* and *CLOSED* list.
- In A* search algorithm, we use search heuristic as well as the cost to reach the node. Hence we can combine both costs as following, and this sum is called as a **fitness number**.

$$f(n) = g(n) + h(n)$$

- Here f(n) refers path length from source node to node n.
- g(n) refers heuristic value of node n.
- A* algorithm finds shortest path in the search space using h(n).
- This algorithm expands less search tree and provides optimal path.

Example 1: Search for node G in the following tree/graph structure using A* algorithm by considering node S as initial state.



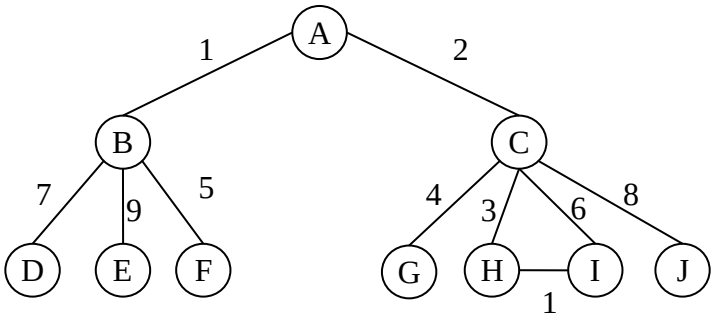
State	h(n)
S	5
A	3
B	4
C	2
D	6
G	0

Open List	Closed List
S_0	---
A_1, G_{10}	S_0
G_{10}, B_7, C_4	S_0, A_1
G_6, B_7, D_{14}	S_0, A_1, C_4
B_7, D_{14}	S_0, A_1, C_4 , G_6

- Here node **G** is GOAL state. So Search becomes Success.
- Final Path Cost is 6 (1 + 1 + 4)
- Optimal path is S → A → C → G

	f(n) = g(n) + h(n)
S → A	f(A) = 1 + 3 → 4
S → G	f(G) = 10 + 0 → 10
S → A → B	f(B) = 3 + 4 → 7
S → A → C	f(C) = 2 + 2 → 4
S → A → C → D	f(D) = 8 + 6 → 14
S → A → C → G	f(G) = 6 + 0 → 6

Example 2: Search for node **I** in the following tree/graph structure using A* algorithm by considering node **A** as initial state.



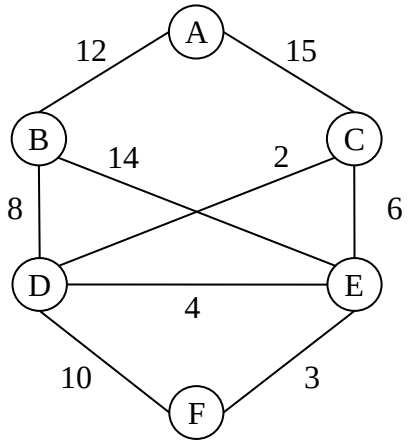
State	h(n)
A	8
B	10
C	4
D	15
E	14
F	12
G	7
H	2
I	0
J	4

Open List	Closed List
A_0	---
B_{11}, C_6	A_0
B_{11}, G_{13}, H_7, I_8, J_{14}	A_0, C_6
B_{11}, G_{13}, I_6, J_{14}	A_0, C_6, H_7
B_{11}, G_{13}, J_{14}	A_0, C_6, H_7, I_6

- Here node **I** is GOAL state. So Search becomes Success.
- Final Path Cost is 6 (2 + 3 + 1)
- Optimal path is A → C → H → I

	f(n) = g(n) + h(n)
A → B	f(B) = 1 +10 → 11
A → C	f(C) = 2 +4 → 6
A → C → G	f(G) = 6 + 7 → 13
A → C → H	f(H) = 5 + 2 → 7
A → C → I	f(I) = 8 + 0 → 8
A → C → J	f(J) = 10 + 4 → 14
A → C → H → I	f(I) = 6 + 0 → 6

Example 3: Search for node **F** in the following tree/graph structure using A* algorithm by considering node **A** as initial state.



State	h(n)
A	0
B	4
C	7
D	5
E	2
F	4

Open List	Closed List
A_0	---
B_{16}, C_{22}	A_0
C_{22}, D_{25}, E_{28}	A_0, B_{16}
D_{22}, E_{23}	A_0, B_{16}, C_{22}
E_{23}, F_{31}	A_0, B_{16}, C_{22}, D_{22}
F_{28}	A_0, B_{16}, C_{22}, D_{22}, E_{23}
---	A_0, B_{16}, C_{22}, D_{22}, E_{23}, F_{28}

- Here node **F** is GOAL state. So Search becomes Success.
- Final Path Cost is **24** (15 + 2 + 4 + 3)
- Optimal path is A → C → D → E → F

	f(n) = g(n) + h(n)
A → B	f(B) = 12 + 4 → 16
A → C	f(C) = 15 + 7 → 22
A → B → D	f(D) = 20 + 5 → 25
A → B → E	f(E) = 26 + 2 → 28
A → C → D	f(D) = 17 + 5 → 22
A → C → E	f(E) = 21 + 2 → 23
A → C → D → E	f(E) = 21 + 2 → 23
A → C → D → F	f(F) = 27 + 4 → 31
A → C → D → E → F	f(F) = 24 + 4 → 28

Algorithm of A* search

1. Place the starting node in the OPEN list.
2. Check if the OPEN list is empty or not, if the list is empty then return failure and stop.
3. else select a node **n** from the OPEN list which has the smallest value of evaluation function f(n).
4. If node **n** is goal node then return success and stop.
5. Otherwise expand node **n** and generate all of its successors, and put node **n** into the closed list.
   (a) For each successor **n'**, check whether **n'** is already in the OPEN list, if not then compute evaluation function for **n'** and place into OPEN list.
   (b) Else if node **n'** is already in OPEN list, then it should be attached to the back pointer which reflects the lowest g(n') value.
6. Goto Step 2.

7.1 Advantages

- A* search algorithm is the best algorithm than other search algorithms.
- A* search algorithm is optimal and complete.
- This algorithm can solve very complex problems.

7.2 Disadvantages

- It does not always produce the shortest path as it mostly based on heuristics and approximation.
- The main drawback of A* is memory requirement as it keeps all generated nodes in the memory, so it is not practical for various large-scale problems.

8. AO* ALGORITHM

- Best-first search is what the AO* algorithm does. The AO* method **divides** any given difficult **problem into a smaller group** of problems that are then resolved **using the AND-OR** graph concept.
- AND OR graphs are specialized graphs that are used in problems that can be divided into smaller problems as shown in the following figure 5.
- The AND side of the graph represents a set of tasks that must be completed to achieve the main goal, while the OR side of the graph represents different methods for accomplishing the same main goal.

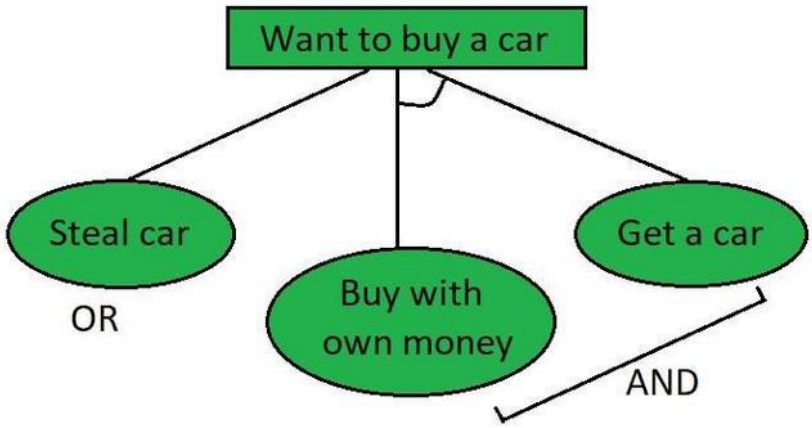


Figure 2.5: AND-OR Graph

- In the above figure, the **buying of a car** may be broken down into smaller problems or tasks that can be accomplished **to achieve the main goal** in the above figure, which is an example of a simple AND-OR graph.
- The other task is to either steal a car that will help us accomplish the main goal or use your own money to purchase a car that will accomplish the main goal.
- The AND symbol is used to indicate the AND part of the graphs, which refers to the need that all subproblems containing the AND to be resolved before the preceding node or issue may be finished.
- The start state and the target state are already known in the knowledge-based search strategy known as the **AO* algorithm**, and the best path is identified by heuristics.

- The informed search technique considerably reduces the algorithm's **time complexity**. The AO* algorithm is far more effective in searching AND-OR trees **than** the A* algorithm.

Working of AO* algorithm:

The evaluation function in AO* looks like this:

f(n) = g(n) + h(n)

f(n) = Actual cost + Estimated cost

here,

f(n) = The actual cost of traversal.

g(n) = the cost from the initial node to the current node.

h(n) = estimated cost from the current node to the goal state.

Algorithm: AO*

1. Let *GRAPH* consist only of the node representing the initial state. (Call this node *INIT.)* Compute *h'(INIT)*

2. Until *INIT* is labeled *SOLVED* or until *INIT's h'* value becomes greater than *FUTILITY,* repeat the following procedure:

(a) Trace the labeled arcs from *INIT* and select for expansion one of the nodes as yet unexpanded that occurs on this path. Call the selected node as ***N***.

(b) Generate the successors of node ***N***. If there is no successors of node ***N***, then assign *FUTILITY* as the *h'* value of node ***N***. This is equivalent to saying that node ***N*** is not solvable. If there are successors, then for each *SUCCESSOR* do the following:

(i) Add *SUCCESSOR* to *GRAPH,*

(ii) If *SUCCESSOR* is a terminal node, label it *SOLVED* and assign it an *h'* value of 0.

(iii) If *SUCCESSOR* is not a terminal node, compute its *h'* value.

(c) Propagate the newly discovered information up the graph by doing the following: Let *S* be a set of nodes that have been labeled *SOLVED* or whose *h'* values have been changed and so need to have values propagated back to their parents. Initialize **S** to node ***N***. Until ***S*** is empty, repeat the, following procedure:

(i) Select a node from ***S***, *c*all this node as *CURRENT Node* and remove it from ***S***.

(ii) Compute the cost of each arcs coming from *CURRENT Node.* The cost of each arc is equal to the sum of the *h'* values of each of the nodes at the end of the arc plus whatever the cost of the arc itself is. Assign as *CURRENT Node's* new *h'* value as the minimum of the costs just computed for the arcs coming from it.

(iii) Mark the best path out of *CURRENT Node* by marking the arc that had the minimum cost as computed in the previous step.

(iv) Mark *CURRENT Node as SOLVED* if all of the nodes connected to it through the new labeled arc have been labeled *SOLVED*.

(v) If *CURRENT Node* has been labeled *SOLVED* or if the cost of *CURRENT Node* was just changed, then its new status must be propagated back up the graph. So add all of the ancestors of *CURRENT Node* to ***S***.

Difference between the A* Algorithm and AO* algorithm

- A* algorithm and AO* algorithm both works on the **best first search**.
- They are both **informed search** and works on given heuristics values.
- **A*** always **gives** the **optimal solution** but AO* doesn't guarantee to give the optimal solution.
- Once AO* got a solution **doesn't explore** all possible paths but A* explores all paths.
- When compared to the A* algorithm, the AO* algorithm uses **less memory.**
- Opposite to the A* algorithm, the AO* algorithm cannot go into an endless **loop.**

9. PROBLEM REDUCTION

- Problem reduction is a technique used to break down a complex problem into smaller, more manageable sub-problems.
- The idea is to solve each sub-problem individually and then combine their solutions to solve the original problem.
- This method is commonly used in problem-solving strategies, particularly in areas like search algorithms, automated reasoning, and planning.

Key Concepts of Problem Reduction

1. **Decomposition**:
    - The original problem is decomposed into several smaller sub-problems. Each sub-problem should be easier to solve than the original problem.
2. **Recursion**:
    - Problem reduction often involves recursion, where each sub-problem is further reduced until a base case is reached. The base case is a sub-problem that can be solved directly without further reduction.
3. **And-Or Graphs**:
    - Problem reduction can be represented using **And-Or graphs**, where nodes represent sub-problems, and edges represent the decomposition.

- **AND nodes**: All child nodes must be solved to solve the parent node.
- **OR nodes**: Solving any one of the child nodes solves the parent node.

4. **Backtracking**:
    - If a particular path in the problem reduction does not lead to a solution, backtracking is used to explore alternative paths or different ways of reducing the problem.

5. **Divide and Conquer**:
    - Problem reduction is closely related to the divide-and-conquer approach, where a problem is divided into independent sub-problems, solved separately, and combined to form the solution.

Algorithm: Problem Reduction

1. Initialize the graph to the starting node.
2. Loop until the starting node is labeled *SOLVED* or until its cost goes above *FUTILITY:*

(a) Traverse the graph, starting at the initial node and following the current best path, and accumulate the set of nodes that are on that path and have not yet been expanded or not labeled as solved.

(b) Pick one of these unexpanded nodes and expand it. If there are no successors, assign *FUTILITY* as the value of this node. Otherwise, add its successors to the graph and for each of them compute *f'(n)*. If of any node is 0, mark that node as *SOLVED.*

(c) Change the *f'(n)* estimate of the newly expanded node to reflect the new information provided by its successors. Propagate this change backward through the graph. If any node contains a successor arc whose descendants are all solved, label the node itself as *SOLVED.*

(d) At each node that is visited while going up the graph, decide which of its successor arcs is the most promising and mark it as part of the current best path.

- The problem is that the choice of which node to expand next must depend not only on the *f'(n)* value of that node but also on whether that node is part of the current best path from the initial node.
- The tree shown in Figure 2.6 makes this even clearer. The most promising single node is G with an *f'(n)* value of 3. It is even part of the most promising arc G-H, with a total cost of 9. But that arc is not part of the current best path since to use it we must also use the arc I-J, with a cost of 27.
- The path from A, through B, to E and F is better, with a total cost of 18. So we should not expand G next; rather we should examine either E or F.

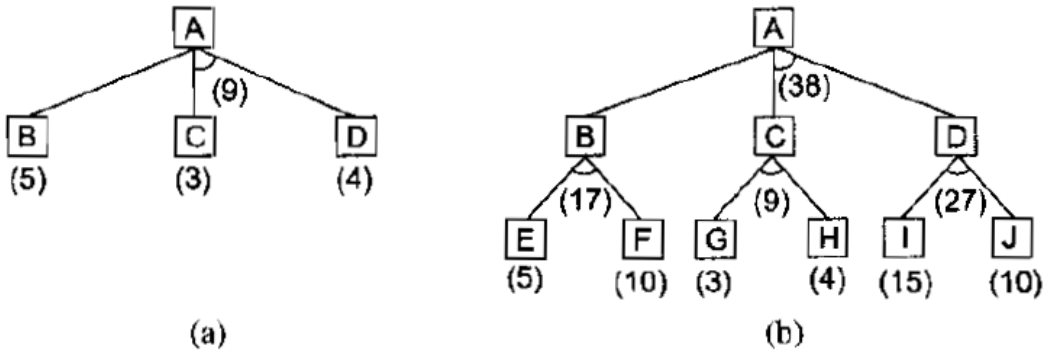


Figure 2.6: *AND-OR Graphs*

Module 3: Game Playing

10. GAME PLAYING-ADVERSIAL SEARCH

- The Adversarial search is a well-suited approach in a competitive environment, where two or more agents have conflicting goals.
- The adversarial search can be employed in two-player zero-sum games which means what is good for one player will be the misfortune for the other. In such a case, there is **no win-win outcome**.
- In artificial intelligence, adversarial search plays a vital role in decision-making, particularly in competitive environments associated with games and strategic interactions.
- By employing adversarial search, AI agents can make optimal decisions while anticipating the actions of an opponent with their opposing objectives.
- It aims to establish an effective decision for a player by considering the possible moves and the counter-moves of the opponents.

The adversarial search in competitive environments can be utilized in the below scenarios where the AI system can assist in determining the best course of action by both considering the possible moves and counter-moves of the opponents.

- Each agent seeks to boost their utility or minimize their loss.
- One agent's action impacts the outcomes and objectives of the other agents.
- Additionally, strategic uncertainty arises when the agents may lack sufficient information about each other's strategies.
- Game search will be implement using the following two techniques.

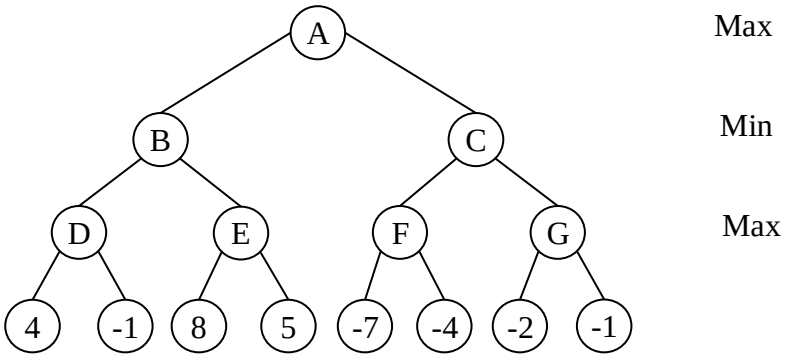
  1. Minimax Search Procedure

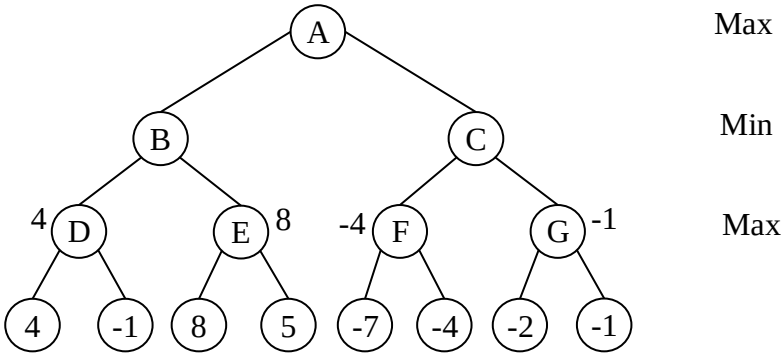
  2. Adding Alpha-Beta Cutoffs

10.1 The Minimax Search Procedure

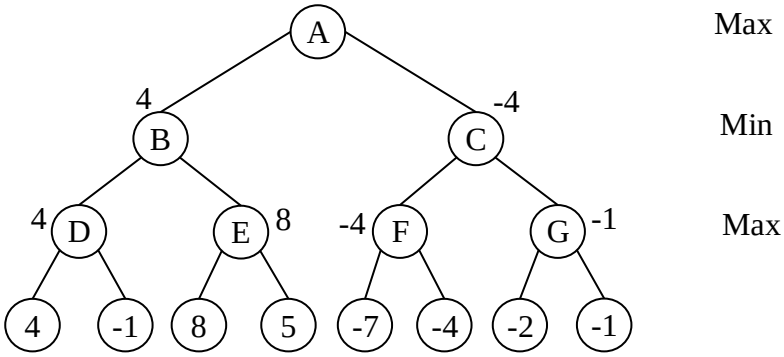
- The *minimax* search procedure is a depth-first, depth-limited search procedure. The idea is to start at the current position and use the plausible-move generator to generate the set of possible successor positions.
- Mini-max algorithm is a recursive or backtracking algorithm which is used in decision-making and game theory.
- It provides an optimal move for the player assuming that opponent is also playing optimally.
- Min-Max algorithm is mostly used for game playing in AI. Such as Chess, Checkers, tic-tac-toe, etc.
- This Algorithm computes the minimax decision for the current state.
- In this algorithm two players play the game, one is called MAX and other is called MIN.
- Both the players fight it as the opponent player gets the minimum benefit while they get the maximum benefit.
- Both Players of the game are opponent of each other, where MAX will select the maximized value and MIN will select the minimized value.
- The minimax algorithm performs a depth-first search algorithm for the exploration of the complete game tree.
- The minimax algorithm proceeds all the way down to the terminal node of the tree, then backtrack the tree as the recursion.
- Max will try to maximize its utility. i.e. Best moves.
- Min will try to minimize its utility. i.e. worst move.

Example 1: Obtain optimal path using minimax search on the following tree/graph structure.

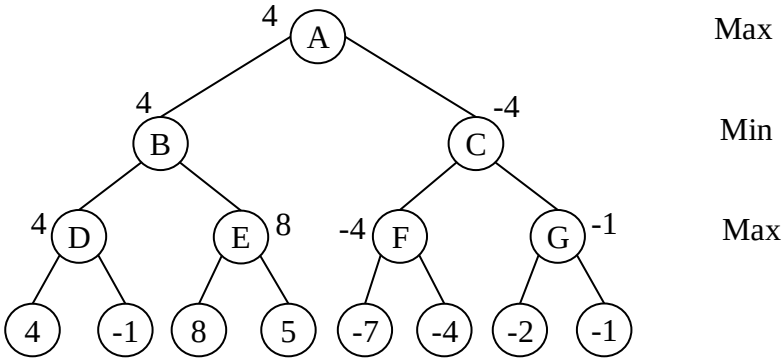




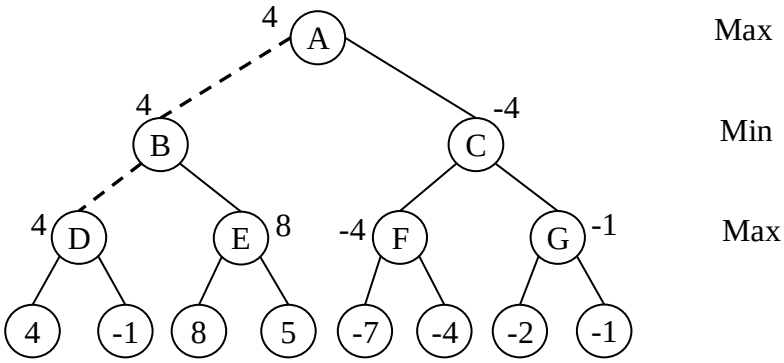
Maximizing player



Minimizing player



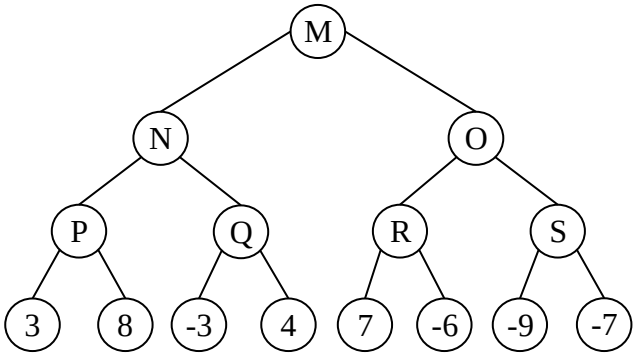
Maximizing player



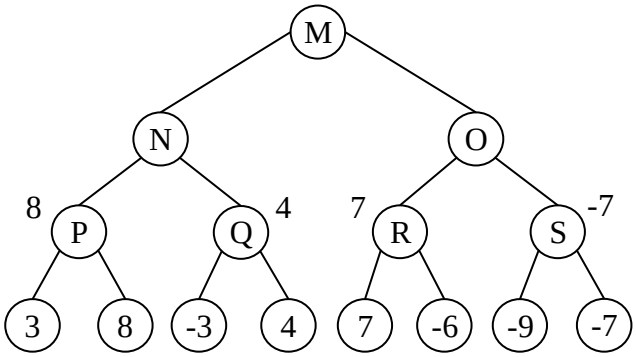
Optimal path is A – B – D

Optimal path value is 4

Example 2: Obtain optimal path using minimax search on the following tree/graph structure.

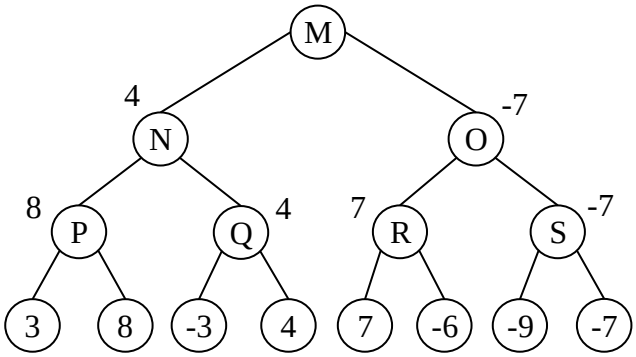


Max
Min
Max



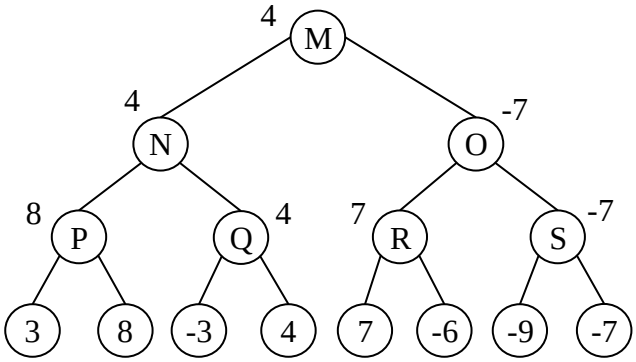
Max
Min
Max

Maximizing player



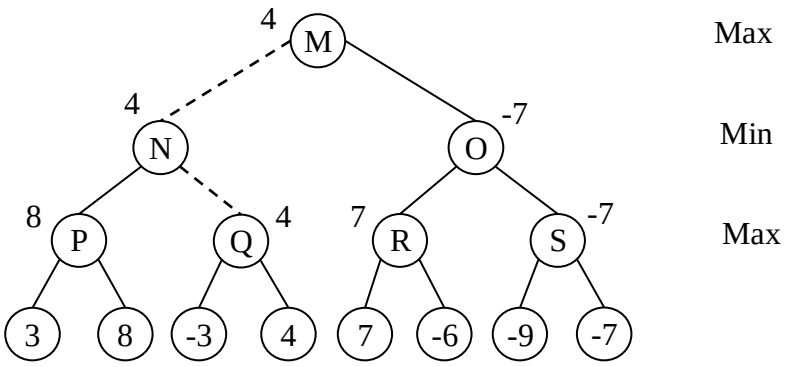
Max
Min
Max

Minimizing player



Max
Min
Max

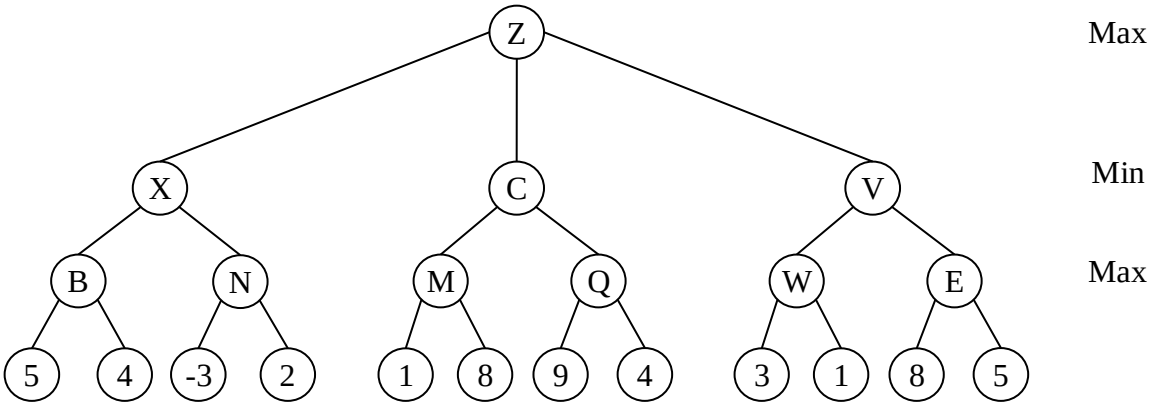
Maximizing player

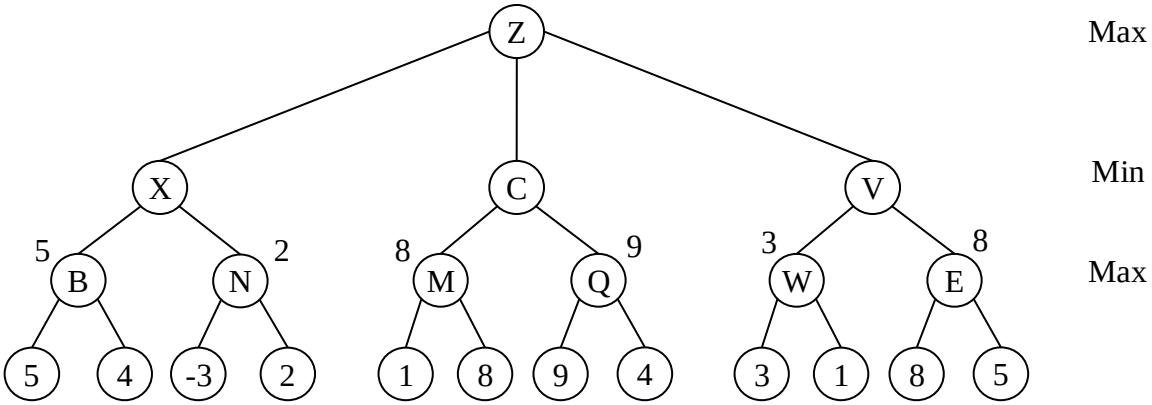


Optimal path is M – N – Q

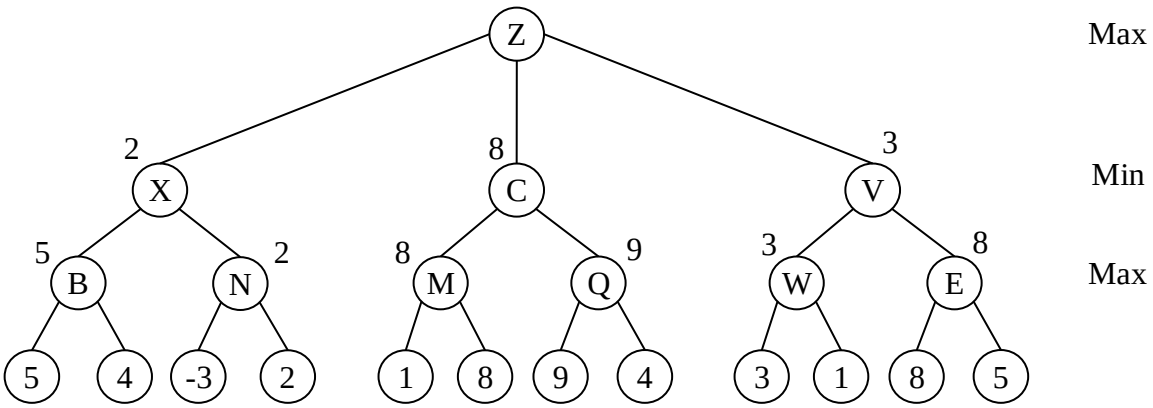
Optimal path value is 4

Example 3: Obtain optimal path using minimax search on the following tree/graph structure.

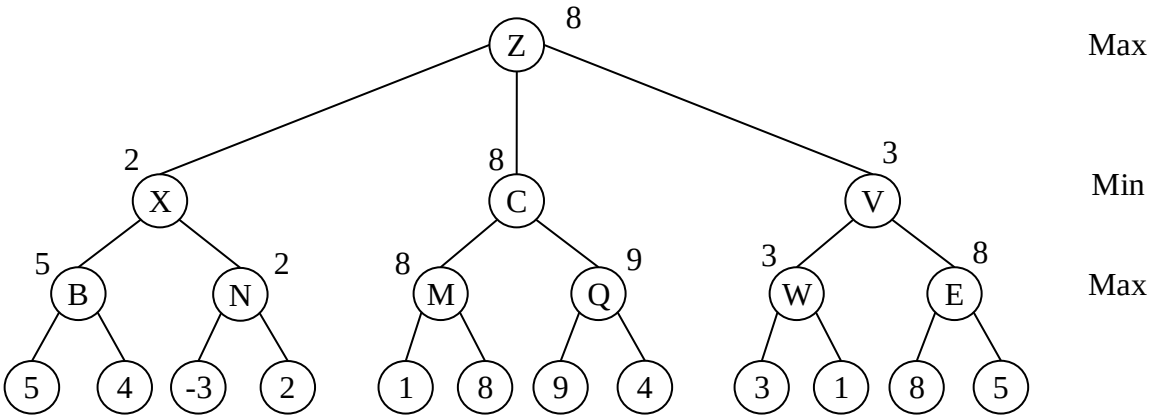




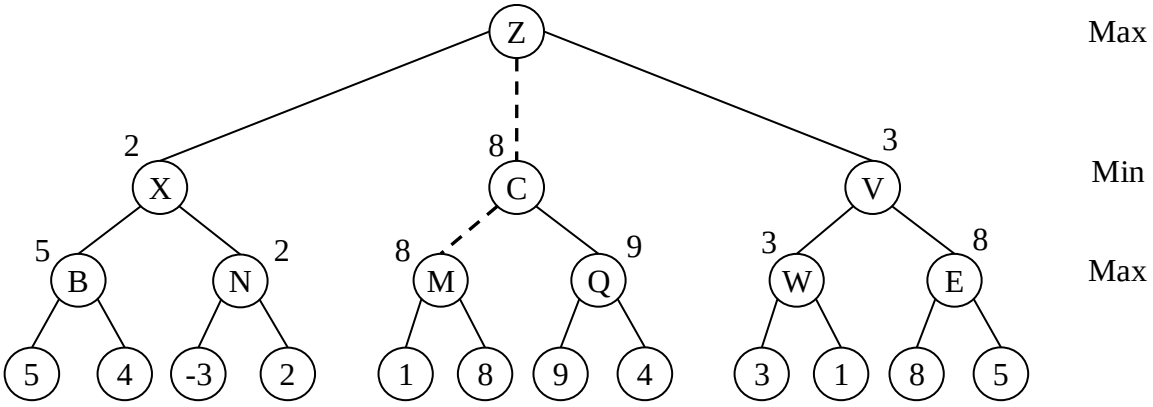
Maximizing player



Minimizing player



Maximizing player



Optimal path is Z – C – M

Optimal path value is 8

Algorithm for MiniMax

```
function MiniMax(node, depth, maximizingPlayer) is
    if  depth == 0 or node is a terminal node then
        return static evaluation of node
    if  MaximizingPlayer then                    // for Maximizer Player
        maxEva = -∞
        for each child of node do
            eva = MiniMax(child, depth+1, false)
            maxEva= max(maxEva,eva)              //gives Maximum of the values
        return maxEva

    else                                         // for Minimizer player
        minEva = +∞
        for each child of node do
            eva = MiniMax(child, depth+1, true)
            minEva= min(minEva, eva)             //gives minimum of the values
        return minEva
```

10.2 Adding Alpha-Beta Cutoffs (Alpha-Beta Pruning)

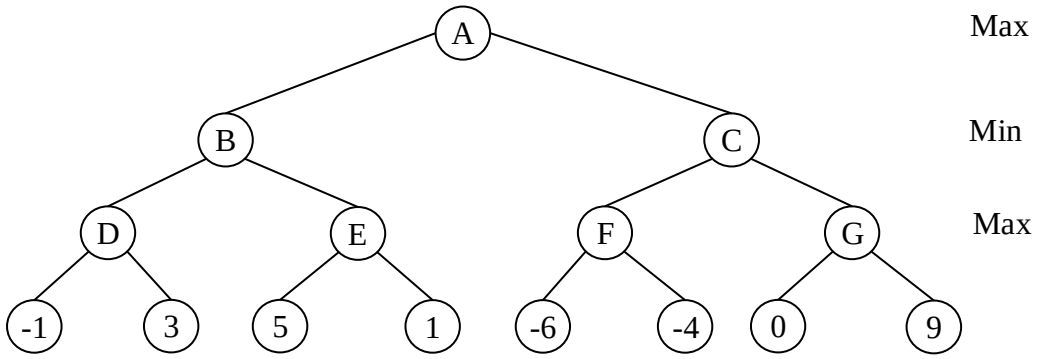
- Alpha-beta pruning is a modified version of the MiniMax algorithm. It is an optimization technique for the MiniMax algorithm.
- As we have seen in the MiniMax search algorithm that the number of game states it has to examine are exponential in depth of the tree. Since we cannot eliminate the exponent, but we can cut it to half. Hence there is a technique by which without checking each node of the game tree we can compute the correct MiniMax decision, and this technique is called **pruning**. This involves two threshold parameter Alpha and beta for future expansion, so it is called **alpha-beta pruning**.
- Alpha-beta pruning can be applied at any depth of a tree, and sometimes it not only prune the tree leaves but also entire sub-tree.
- The two-parameter can be defined as:

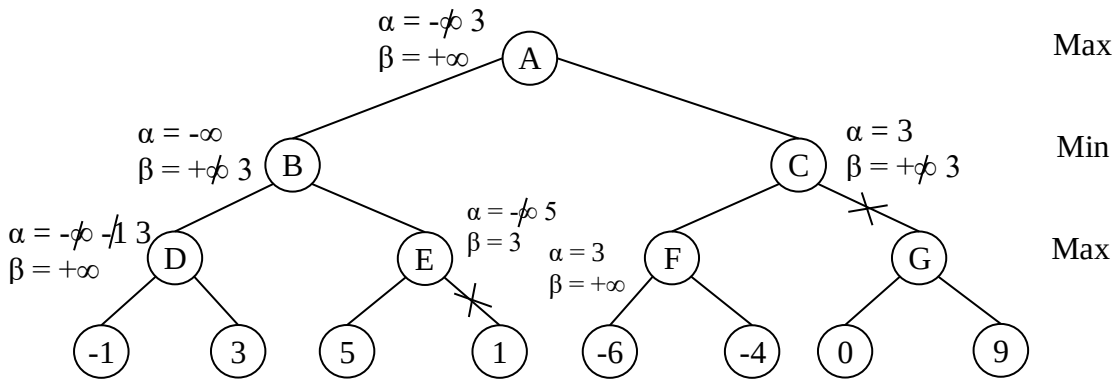
  Alpha denotes MAX value. Worst case value is -∞.

  Beta denotes MIN value. Worst case value is +∞.
- At MAX level alpha value will be update, beta value will remain same.
- At MIN level alpha value will remain same, beta value will be update.
- This algorithm follows DFS strategy.

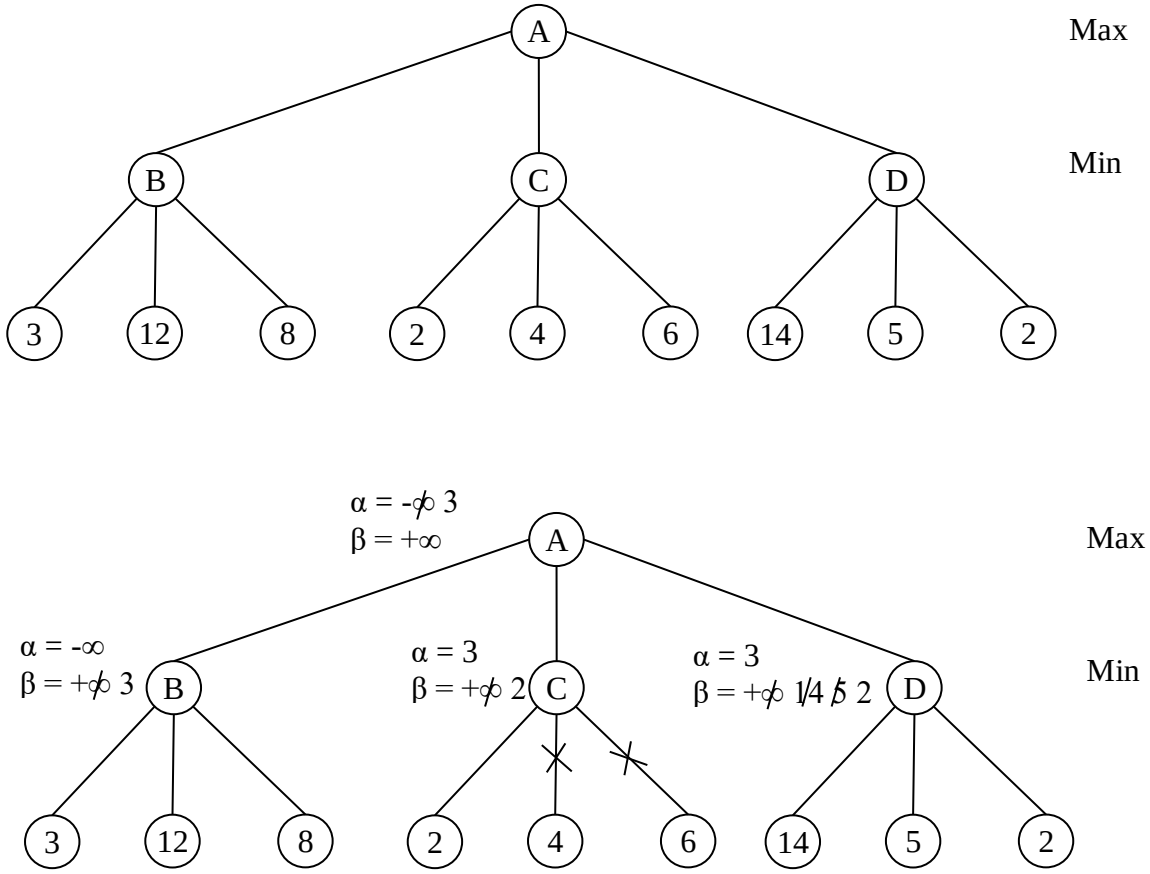
- Here search process is to handle both maximizing and minimizing player, it includes two bounds, one for each player. i.e. lower bound and upper bound.
- Every time this algorithm will verify α>=β. If this condition is true, prune the remaining sub-tree.
- Whenever we are moving from top to bottom, i.e. from parent to child node, parent node values will be transfer to child node.
- Whenever we are moving from bottom to top, i.e. from child to parent node, *parent node values will be update based on its child node*.
- Alpha value will be update if parent node is at MAX level, Beta value will be update if parent node is at MIN level.

Example 1: Apply Alpha-Beta Pruning on the following tree/graph structure.

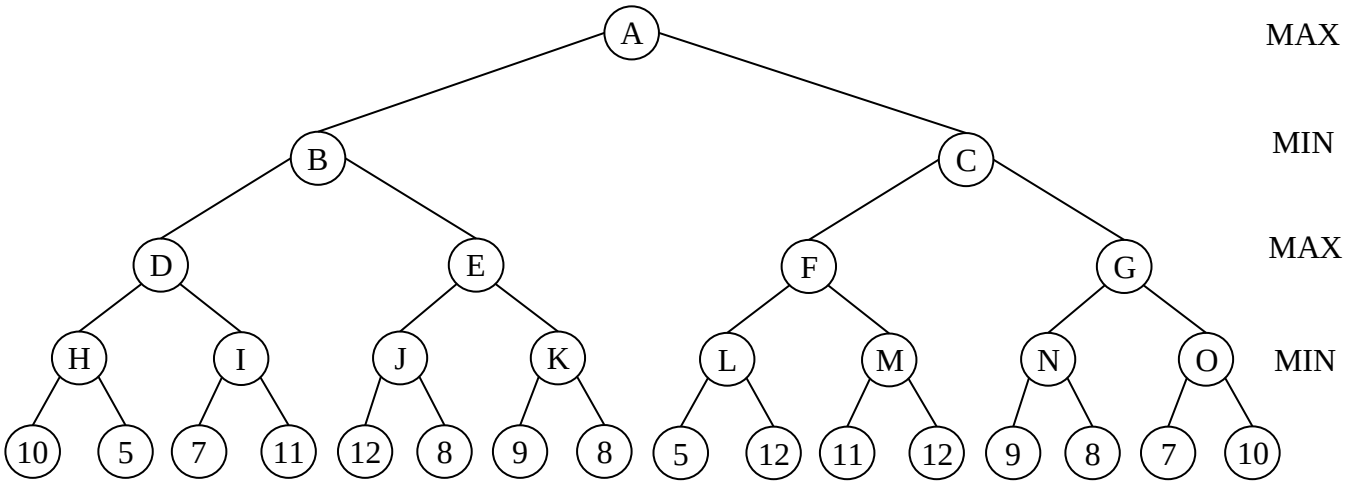


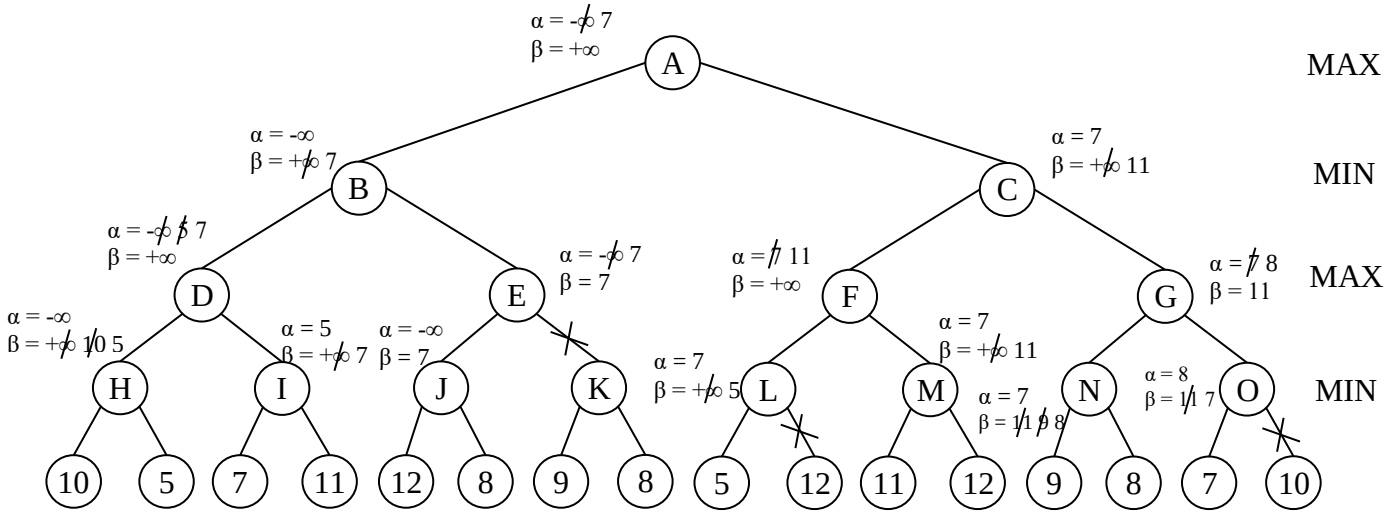


Example 2: Apply Alpha-Beta Pruning on the following tree/graph structure.

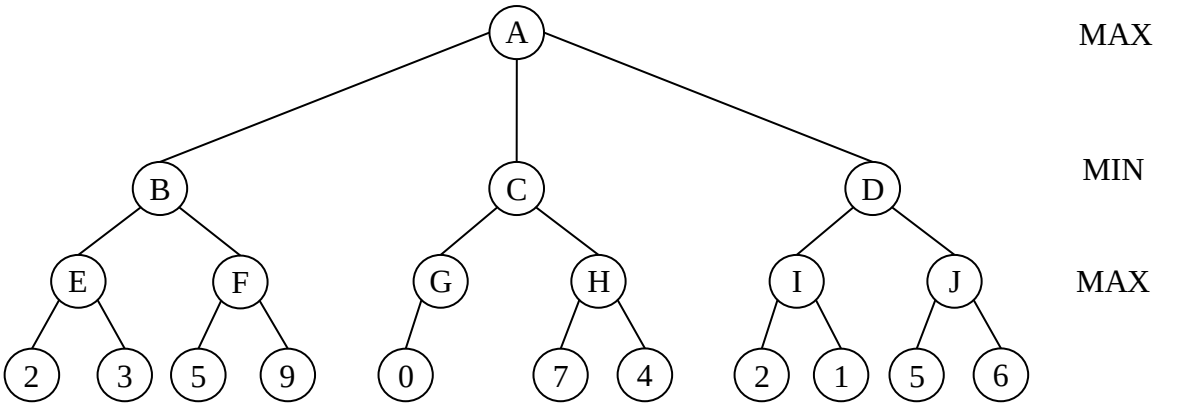


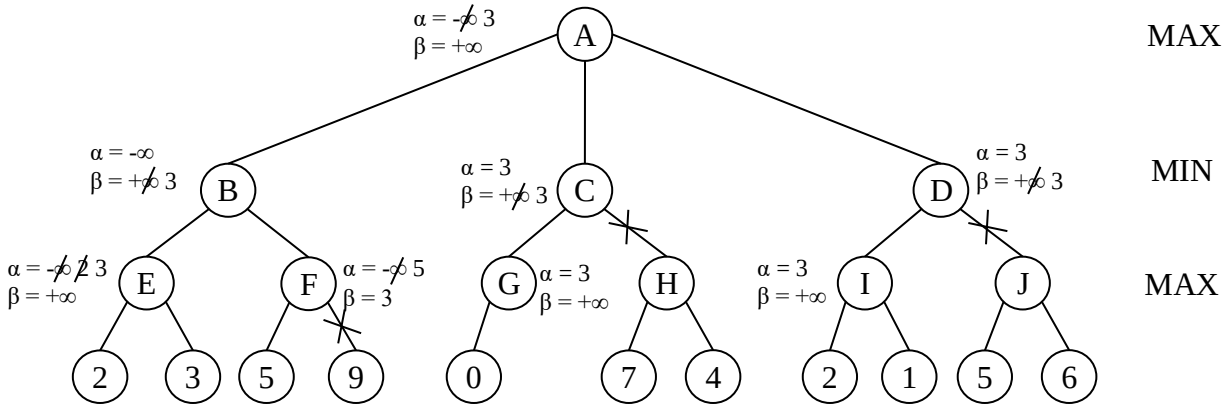
Example 3: Apply Alpha-Beta Pruning on the following tree/graph structure.



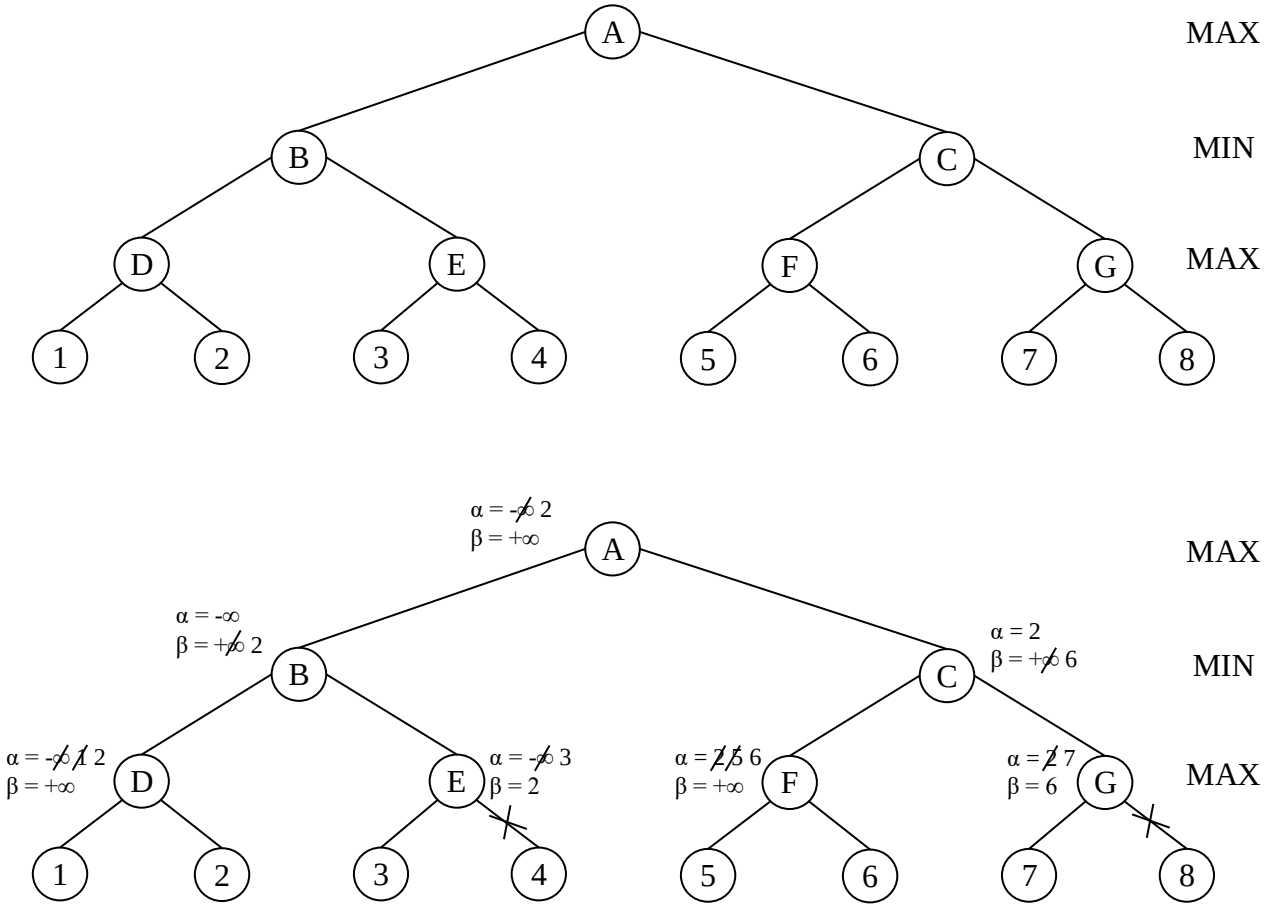


Example 4: Apply Alpha-Beta Pruning on the following tree/graph structure.





Example 5: Apply Alpha-Beta Pruning on the following tree/graph structure.



Algorithm Alpha-Beta Pruning

```
function MiniMax (node, depth, alpha, beta, maximizingPlayer) is
    if  depth == 0 or node is a terminal node then
        return static evaluation of node
    if  MaximizingPlayer then                    // for Maximizer Player
        maxEva = - ∞
        for each child of node do
            eva = MiniMax(child, depth+1, alpha, beta, False)
            maxEva = max(maxEva, eva)
            alpha = max(alpha, maxEva)
            if alpha >= beta
                break
        return maxEva
```

```
else                                    // for Minimizer player
        minEva= +∞
        for each child of node do
                eva = MiniMax(child, depth+1, alpha, beta, true)
                minEva = min(minEva, eva)
                beta = min(beta, minEva)
                if alpha >= beta
                        break
        return minEva
```

10.3 Optimal decisions in multiplayer games

- Many popular games allow more than two players. Let us examine how to extend the minimax idea to multiplayer games.
- First, we need to replace the single value for each node with a *vector* of values. For example, in a three-player game with players A, B, and C, a vector $<V_A, V_B, V_C>$ is associated with each node.
- For terminal states, this vector gives the utility of the state from each player's viewpoint.
- The simplest way to implement this is to have the UTILITY function return a vector of utilities.
- Now we have to consider non terminal states. Consider the node marked X in the game tree shown in Figure 2.7. In that state, player C chooses what to do. The two choices lead to terminal states with utility vectors $<V_A =1, V_B =2, V_C =6>$ and $<V_A =4, V_B =2, V_C =3>$. Since 6 is bigger than 3, **C** should choose the first move.
- This means that if state **X** is reached, subsequent play will lead to a terminal state with utilities $<V_A =1, V_B =2, V_C =6>$. Hence, the backed-up value of **X** is this vector.
- The backed-up value of a node **n** is always the utility vector of the successor state with the highest value for the player choosing at **n**.
- Anyone who plays multiplayer games, such as Diplomacy, quickly becomes aware that much more is going on than in two-player games.

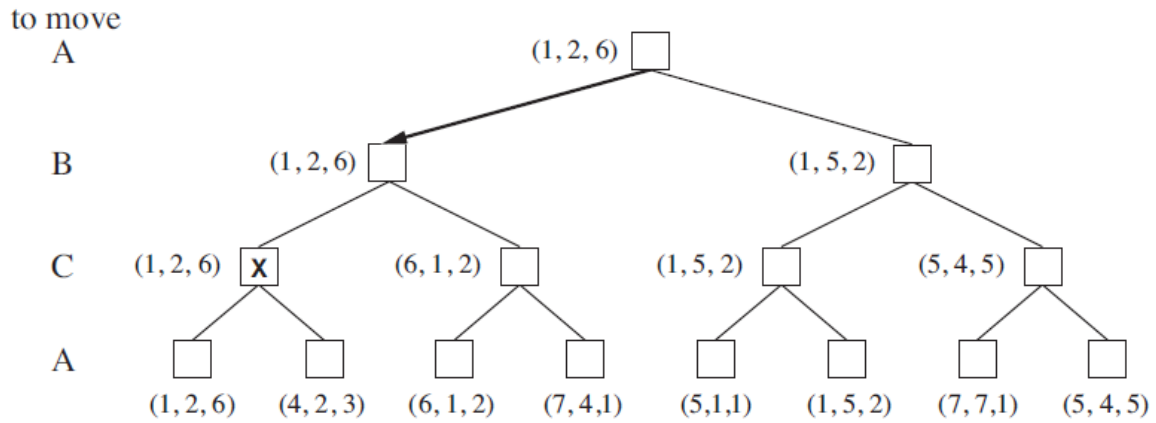


Figure 2.7: The first three plies of a game tree with three players (A, B, C). Each node is labeled with values from the viewpoint of each player. The best move is marked at the root.

10.4 Problem in Game playing

10.4.1 Lack of Control

- The integration of AI systems and machine learning can result in unpredictable outcomes. These systems learn and adapt in real-time based on player behavior and other game world factors.
- This makes it a challenge for developers to predict and control every aspect of the game. This lack of control can sometimes detract from the intended gaming experience, with the AI behaving in ways that weren't originally intended by the creators.

10.4.2 Lack of Creativity

- Procedural generation and generative AI tools are now frequently used for content creation in games. These AI tools can create huge amounts of game content quickly, which may seem like a good thing for gaming companies.
- However, while procedural generation can provide endless environment and scenarios, it might lack the depth and engaging gaming narratives that a human designer could offer. There's a risk that relying too heavily on AI for content creation can result in repetitive and less engaging gaming experiences.
- If every game design company is using the same tools, there won't be as much variety in video games.

10.4.3 Technical Issues

- Aspects of game development that heavily rely on AI can introduce a host of technical problems.
- For instance, while AI tools can facilitate aspects like real-time adjustments, they also demand more from the game engine in terms of computational power.

- Bugs and unforeseen interactions within advanced AI can lead to breakdowns in game play or even crash the entire game. Ensuring smooth integration of AI into the gaming experience requires rigorous testing and refinement.

10.5 Evaluation functions

- An evaluation function returns an *estimate* of the expected utility of the game from a given position, just as the heuristic functions return an estimate of the distance to the goal.
- The performance of a game-playing program depends strongly on the quality of its evaluation function.
- An inaccurate evaluation function will guide an agent toward positions that turn out to be lost.
- How exactly do we design good evaluation functions?
- First, the evaluation function should order the *terminal* states in the same way as the true utility function. Otherwise, an agent using the evaluation function might error even if it can see ahead all the way to the end of the game.
- Second, the computation must not take too long.
- Third, for non-terminal states, the evaluation function should be strongly correlated with the actual chances of winning.
- Most evaluation functions work by calculating various **features** of the state—for example, in chess, we would have features for the number of white pawns, black pawns, white queens, black queens, and so on.
- The features, taken together, define various *categories* or *equivalence classes* of states: the states in each category have the same values for all the features.
- A reasonable evaluation for states in the category is termed as expected value.

QUESTION BANK

1 Mark: Objective Questions

Module 1: Uninformed Search

1. When should Depth-First Search be preferred over Breadth-First Search? [BL:2]

(A) When the search space is very large and memory is limited

(B) When the solution is close to the root

(C) When the search tree is shallow

(D) When there are multiple solutions

Answer: (A) When the search space is very large and memory is limited

2. In Depth First Search, how many times a node is visited?

(A) Once

(B) Twice

(C) Equivalent to number of indegree of the node

(D) Thrice

Answer: (A) Once

3. Which search strategy always expands the shallowest unexpanded node first? [BL:1]

(A) Depth-First Search

(B) Breadth-First Search

(C) Uniform-Cost Search

(D) Iterative Deepening Search

Answer: (B) Breadth-First Search

4. Which is true regarding BFS (Breadth First Search)?

(A) BFS will get trapped exploring a single path

(B) The entire tree so far been generated must be stored in BFS

(C) BFS is not guaranteed to find a solution if exists

(D) BFS is nothing but Binary First Search

Answer: (B) The entire tree so far been generated must be stored in BFS

5. Which of the following search strategies uses a queue to store the nodes?

(A) Depth-First Search

(B) Breadth-First Search

(C) Hill Climbing

(D) A* Search

Answer: (B) Breadth-First Search

Module 2: Informed Search

6. What is the other name of informed search strategy?

(A) Simple search

(B) Heuristic search

(C) Online search

(D) None of the mentioned

Answer: (B) Heuristic search

7. Which is used to improve the performance of heuristic search?

(A) Quality of nodes

(B) Quality of heuristic function

(C) Simple form of nodes

(D) None of the mentioned

Answer: (B) Quality of heuristic function

8. How can the effectiveness of the Hill Climbing algorithm be improved in solving optimization problems? [BL:2]

(A) By restarting the algorithm multiple times from different initial points

(B) By increasing the depth of the search

(C) By using a deeper heuristic

(D) By implementing Alpha-Beta pruning

Answer: (A) By restarting the algorithm multiple times from different initial point

9. Which of the following search strategies can get stuck in local maxima?

(A) A* Search

(B) Breadth-First Search

(C) Hill Climbing

(D) Depth-First Search

Answer: (C) Hill Climbing

10. In AI, what does the 'h' function represent in the A* search algorithm? [BL:2]

(A) Total path cost

(B) Heuristic estimate of the cost to reach the goal

(C) The branching factor

(D) None of the above

Answer (B) Heuristic estimate of the cost to reach the goal

11. In an A* search, if the heuristic function overestimates the cost to reach the goal, what is the consequence? [BL:3]

(A) The search may find a suboptimal solution

(B) The search will always find the optimal solution

(C) The search may terminate early

(D) The search will be faster

Answer: (A) The search may find a suboptimal solution

12. Which search is complete and optimal when h(n) is consistent?

(A) Best-first search

(B) Depth-first search

(C) Both Best-first & Depth-first search

(D) A* search

Answer: (D) A* search

13. In the context of heuristic search, which algorithm guarantees finding the shortest path to the goal if the heuristic is admissible?

(A) Depth-First Search

(B) Breadth-First Search

(C) A* Search

(D) Hill Climbing

Answer: (C) A* Search

Module 3: Game Playing

14. The Mini-Max algorithm is used in:

(A) Uninformed Search

(B) Adversarial Search

(C) Heuristic Search

(D) Problem Reduction

Answer: (B) Adversarial Search

15. Why is Alpha-Beta pruning used in game playing? [BL:3]

(A) To find the optimal solution faster

(B) To prune unneeded branches in the search tree

(C) To optimize memory usage

(D) To simplify the evaluation function

Answer: (B) To prune unneeded branches in the search tree

16. In the context of game playing, which algorithm is primarily used for making decisions in multiplayer games? [BL:3]

(A) Hill Climbing

(B) A* Algorithm

(C) Mini-Max Algorithm

(D) Genetic Algorithm

Answer: (C) Mini-Max Algorithm

17. Which value is assigned to alpha and beta in the alpha-beta pruning?

(A) Alpha = max

(B) Beta = min

(C) Beta = max

(D) Both Alpha = max & Beta = min

Answer: (D) Both Alpha = max & Beta = min

18. What is the main purpose of Alpha-Beta pruning in game playing?

(A) To improve the accuracy of the heuristic function

(B) To reduce the number of nodes evaluated in the mini-max algorithm

(C) To find the shortest path in an unweighted search problem

(D) To solve problems with multiple goals

Answer: (B) To reduce the number of nodes evaluated in the mini-max algorithm

19. Which function is used to calculate the feasibility of whole game tree?

(A) Evaluation function

(B) Transposition

(C) Alpha-beta pruning

(D) All of the mentioned

Answer: (A) Evaluation function

2 Marks: Short Answer Questions

Module 1: Uninformed Search

1. Define Breadth-First Search (BFS) [BL: 1]
2. Apply the BFS algorithm to find the shortest path in a simple graph.[BL:2]
3. Differentiate DFS and BFS.

Module 2: Informed Search

4. Describe how the Hill Climbing algorithm works. [BL:2]
5. Use the Hill Climbing technique to solve a problem where multiple peaks exist.[BL:2]
6. List the primary characteristics of the A* algorithm [BL: 1]

7. How would you modify the A* algorithm for use in a weighted graph?[BL:4]
8. Analyze the effectiveness of Heuristic search in scenarios with partial information.[BL:3]
9. Differentiate between A* algorithm and AO* algorithm in terms of their approach to problem solving. [BL:2]

Module 3: Game Playing

10. Evaluate the strengths and weaknesses of the mini-max algorithm when dealing with multiplayer games.[BL:3]
11. What role does the evaluation function play in the mini-max algorithm? [BL: 3]
12. Summarize how Alpha-Beta pruning optimizes the mini-max algorithm. [BL:2]

5 Marks : Descriptive Questions

Module 1: Uninformed Search

1. Explain Depth First Search with an example. [BL:2]
2. Write algorithm to implement Breadth First Search. [BL:2]
3. What is the main difference between Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms? [BL:3]

Module 2: Informed Search

4. Given a simple maze with walls and open spaces, apply the Hill Climbing algorithm to find a path from the start position to the goal. Describe your process.[BL:3]
5. Explain how the A* algorithm combines features of both Uniform Cost Search and Greedy Search.[BL:2]
6. With an example explain A* approach. [BL:2]
7. Write algorithm to implement Best-First Search. [BL:2]

Module 3: Game Playing

8. Analyze the strengths and weaknesses of Alpha-Beta Pruning in comparison to the Mini-Max algorithm in game playing.[BL:4]
9. With an example explain Alpha-Beta Pruning. [BL:3]
10. Evaluate the effectiveness of different evaluation functions in assessing the quality of game states in a game like chess.[BL:2]